

Project/Thesis No.:

CSE 4000: Thesis/ Project

A Machine Learning-Based Runtime Data-in-Use Protection Framework with Adaptive Decryption and Exposure Control

By

Faiyaz Mahmud

Roll: 2007103



**Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh
July, 2026**

A Machine Learning-Based Runtime Data-in-Use Protection Framework with Adaptive Decryption and Exposure Control

By

Faiyaz Mahmud

Roll: 2007103

Supervisor:

Dr. Muhammad Aminul Haque Akhand

Professor

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

Signature

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

Khulna 9203, Bangladesh

July, 2026

Acknowledgment

All praise is due to the Almighty for the strength, patience, and opportunity to complete this thesis. I would like to express my deepest gratitude to my supervisor, Dr. Muhammad Aminul Haque Akhand, Professor, Department of Computer Science and Engineering, Khulna University of Engineering & Technology, for the continuous guidance, insightful feedback, and encouragement provided at every stage of this work - from shaping the problem of data-in-use protection, through the design of the adaptive decryption control layer, to the interpretation of the experimental results. I thank the faculty members and staff of the department for the academic environment and resources that made this work possible, and the authors of the related studies cited throughout this report, whose work informed and motivated this framework. Finally, I am grateful to my parents, family, and friends for their unconditional support throughout this undergraduate thesis.

Author

Abstract

Modern application systems commonly protect sensitive data at rest using storage-level or field-level encryption and protect data in transit using transport-layer security. However, encrypted fields are often decrypted automatically the moment an authenticated request retrieves an entity from the persistence layer. This creates a data-in-use exposure risk: if a stolen token, compromised account, malicious insider, or behaviorally anomalous request passes static authorization, sensitive plaintext may be produced before any runtime risk is evaluated. This thesis designs and implements a machine learning-based adaptive decryption control framework for runtime data-in-use protection. The framework re-engineers the Object-Relational Mapper retrieval path of a NestJS/PostgreSQL backend so that ciphertext survives database retrieval and decryption becomes an explicit, policy-gated operation. For each sensitive request, server-controlled behavioral features are extracted: user role, resource sensitivity, office-hour status, record-ownership match, sixty-second request velocity, and twenty-four-hour failed-login count. A seventy-five-tree Random Forest classifier, trained on a reproducible synthetic corpus of 10,000 rule-labeled access events, classifies each request into LOW, MEDIUM, or HIGH risk; the trained model is transpiled into dependency-free JavaScript executing inside the Node.js application process in approximately fifty microseconds. A graduated policy engine maps the predicted risk to full AES-256-GCM decryption, partial decryption with masking, or access denial. The framework is evaluated as a proof-of-concept on a quarantined synthetic test set of 1,500 samples, where the classifier achieved 0.9780 accuracy and 0.9777 weighted F1-score, a 99.14% legitimate pass rate, and 92.8% outright interception of simulated attack vectors. End-to-end prototype verification showed that the same endpoint returns full plaintext, a masked response, or an HTTP 403 denial purely as a function of runtime behavior, and that the denial path performs zero application-level decryption calls. The results indicate that adaptive per-request decryption control is feasible with low runtime overhead, although validation with real access telemetry, distributed deployment, and broader attack scenarios is required before production use.

Contents

Title Page	1
Acknowledgment	ii
Abstract	iii
Contents	iv
List of Tables.....	vii
List of Figures	viii
List of Abbreviations.....	ix
Chapter I Introduction	1
1.1 Background of the Study.....	1
1.2 Problem Statement	3
1.3 Research Gap	4
1.4 Research Objectives	4
1.5 Research Questions	5
1.6 Significance of the Study	5
1.7 Scope of the Study	6
1.8 Thesis Organization	6
Chapter II Literature Review	8
2.1 Background of the Problem	8
2.2 Significance of Machine Learning in Adaptive Access Control.....	10
2.3 Benchmark Dataset and Procedure of Data Collection.....	12
2.4 Review of Previous Studies	12
2.5 Summary of Existing Methods	14
2.6 Identified Research Gap.....	14
2.7 Conceptual Framework and Proposed Research Direction.....	15
2.8 Chapter Summary.....	15
Chapter III Research Methodology.....	16
3.1 Research Framework.....	16
3.2 Data Collection and Synthetic Dataset Generation.....	17
3.3 Data Preprocessing and Feature Generation	20
3.4 Algorithm and Model Development	21
3.5 Experimental Design and Validation Strategy	26

3.6 Evaluation Metrics	26
3.7 Significance of the Proposed Method	27
Chapter IV Experimental Studies	28
4.1 Experimental Environment	28
4.2 Parameter Settings.....	29
4.3 Baseline and Comparative Methods.....	29
4.4 Procedure of Experimentation.....	29
4.5 Experimental Results	30
4.6 Experimental Analysis	35
4.7 Comparative Analysis	36
4.8 Discussion of Findings.....	37
4.9 Risk of Rule-Learning Bias in Synthetic Evaluation	37
4.10 Chapter Summary.....	38
Chapter V Engineering, Ethical, and Professional Considerations	39
5.1 Introduction.....	39
5.2 Intellectual Property Considerations	39
5.3 Ethical Considerations	39
5.4 Data Privacy and Security Considerations.....	40
5.5 Safety Considerations	40
5.6 Legal Considerations.....	41
5.7 Societal, Health, and Cultural Impact	41
5.8 Environmental and Sustainability Considerations	41
5.9 Summary	41
Chapter VI Complex Engineering Problems and Activities	43
6.1 Introduction.....	43
6.2 Complex Engineering Problems Associated with the Project.....	43
6.3 Complex Engineering Activities Associated with the Project	44
6.4 Mapping of the Proposed Work with Complex Engineering Attributes.....	46
6.5 Summary	46
Chapter VII Conclusion and Future Work	47
7.1 Summary of the Study.....	47
7.2 Major Findings.....	47
7.3 Limitations of the Study.....	48

7.4 Future Recommendation	48
7.5 Closing Statement	49
References	50

List of Tables

Table 2.1	Summary of existing approach families and the properties required for adaptive data-in-use protection.....	14
Table 3.1	Marginal feature distributions of the synthetic access-event corpus (10,000 samples, seed 42).	18
Table 3.2	HIGH-risk labeling rules of the dataset generator.	19
Table 3.3	MEDIUM-risk labeling rules of the dataset generator.....	19
Table 3.4	Stratified dataset partitions and their roles.....	19
Table 3.5	The six business features: sources, types, and selection rationale.	20
Table 3.6	Static numerical mapping: the invariant 8-element feature vector shared by training and runtime.	21
Table 3.7	Random Forest hyperparameters and their justification.	22
Table 3.8	Graduated risk-to-policy mapping enforced by the adaptive layer.....	25
Table 4.1	Implementation environment of the framework.	28
Table 4.2	Parameter settings used throughout the reported experiments.....	29
Table 4.3	Per-class classification report on the quarantined test set (single final evaluation).....	30
Table 4.4	Confusion matrix on the test set: 0.86% false-alarm rate on legitimate traffic; 92.8% outright attack interception; zero hard blocks of MEDIUM-tier users..	31
Table 4.5	Learned feature-importance ranking of the Random Forest.	32
Table 4.6	Measured properties of the transpiled model inside the Node.js runtime.....	33
Table 4.7	Transpilation-fidelity scenarios scored through the JavaScript artifact.....	34
Table 4.8	Field-level exposure of the same encrypted record under the three adaptive policies.	35
Table 4.9	Selected model versus the linear baseline under an identical protocol.	36
Table 4.10	Threat-by-threat comparison of the static baseline pipeline and the proposed adaptive framework.....	36
Table 6.1	Mapping of the thesis to Complex Engineering Problem (CEP) attributes.	43
Table 6.2	Mapping of the thesis to Complex Engineering Activity (CEA) attributes.....	44
Table 6.3	Consolidated mapping of major system components to CEP/CEA attributes. .	46

List of Figures

Figure 1.1	Baseline Architecture Showing Data-in-Use Weakness in Typical Enterprise Backends.....	2
Figure 1.2	Analysis of Anomalous Context Attacks Exploiting Static Access Controls...	3
Figure 1.3	Gantt chart showing the week-by-week project plan across the two academic terms of the thesis.	7
Figure 2.1	The three states of data and their protection mechanisms; the data-in-use state addressed by this thesis is highlighted.....	10
Figure 2.2	Concept of machine learning-based automatic risk assessment of access requests.	11
Figure 3.1	Four-stage research framework of the thesis, from dataset engineering to adaptive enforcement in the backend.	16
Figure 3.2	Eight-layer security architecture of the proposed framework; Layer 4 is the thesis contribution.....	17
Figure 3.3	Offline training pipeline and zero-dependency transpilation into the backend runtime.....	23
Figure 3.4	Standard ORM retrieval versus the late-binding cryptographic redesign.	24
Figure 3.5	End-to-end runtime flow of one protected request through the adaptive framework.....	25
Figure 3.6	Risk-to-policy decision flow of the graduated enforcement engine.	26
Figure 4.1	Visualization of the test-set confusion matrix (heatmap) and per-class F1-scores.	31
Figure 4.2	Feature-importance ranking of the trained model.	33

List of Abbreviations

ABAC	Attribute-Based Access Control
AES	Advanced Encryption Standard
ALE	Application-Level Encryption
API	Application Programming Interface
CSE	Computer Science and Engineering
CSV	Comma-Separated Values
GCM	Galois/Counter Mode
IDOR	Insecure Direct Object Reference
IV	Initialization Vector
JWT	JSON Web Token
KUET	Khulna University of Engineering & Technology
LR	Logistic Regression
ML	Machine Learning
ORM	Object-Relational Mapper
PII	Personally Identifiable Information
RBAC	Role-Based Access Control
RF	Random Forest
SQL	Structured Query Language
TDE	Transparent Data Encryption
TLS	Transport Layer Security

UUID

Universally Unique Identifier

List of Symbols

$\mathbf{x}$	8-element numeric feature vector fed to the risk model
$\mathbf{x}[i]$	The i -th element of the feature vector (strict index mapping)
c	Predicted risk class, $c \in \{0 \text{ (LOW)}, 1 \text{ (MEDIUM)}, 2 \text{ (HIGH)}\}$
$\mathbf{P}(c \mathbf{x})$	Class-probability vector produced by the Random Forest
T	Number of decision trees in the forest ($T = 75$)
d	Maximum depth of each decision tree ($d = 12$)
P, R	Precision and recall of a class
$F1$	Harmonic mean of precision and recall, $F1 = 2PR / (P + R)$
$\Delta t(\text{req})$	60-second sliding window for request velocity
$\Delta t(\text{fail})$	24-hour window for failed authentication attempts

CHAPTER I

Introduction

Conventional application systems encrypt sensitive data at rest and in transit but decrypt it unconditionally the moment an authenticated request arrives, leaving data in use unprotected against behaviorally anomalous but technically authorized access. This chapter introduces that data-in-use problem, states the research gap left by static encryption and static access control, and defines the aim, objectives, research questions, significance, scope, limitations, and organization of this thesis.

1.1 Background of the Study

Information security practice organizes the protection of data around its three states: data at rest, data in transit, and data in use. The first two states are served by mature, widely deployed mechanisms. Data at rest is protected by Transparent Data Encryption (TDE) and storage-level encryption, which encrypt database files, backups, and disks; data in transit is protected by Transport Layer Security (TLS). Many organizations additionally apply Application-Level Encryption (ALE), in which individually sensitive fields — bank account numbers, national identity numbers, medical records — are encrypted inside the application before they are ever handed to the database. In combination, these controls ensure that a stolen disk, an exfiltrated backup, a compromised database administrator account, or an SQL-injection read primitive discloses only ciphertext.

The third state — data in use — is the weakest link of ordinary application systems. Consider a typical enterprise backend built on an Object-Relational Mapper (ORM) such as TypeORM. Encrypted columns are declared with value transformers: an encryption function runs when an entity is saved, and a decryption function runs automatically when an entity is loaded. The consequence is subtle but severe: whenever any authenticated request triggers a database read, every encrypted column is decrypted into server memory during entity hydration, before the business layer has had any opportunity to evaluate who is asking, how often, from what temporal context, and for whose records. Decryption is an implicit side effect of a query rather than a deliberate security decision.

Static access control does not repair this weakness. Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC) answer the question "may this principal call

this endpoint?" once, at the gate, and their verdict is binary. A customer reading their own record for the fortieth time in one minute receives exactly the same fully decrypted response as a customer reading it once; an account that just survived a burst of failed login attempts is indistinguishable from one with a clean history. Attacks such as Insecure Direct Object Reference (IDOR) enumeration, high-velocity data scraping, and credential-stuffing-based account takeover are precisely the attacks that defeat static controls while exhibiting strongly anomalous runtime behavior.

Machine learning offers a natural instrument for this problem. Security rules of the form "IF the requester is a customer AND the record belongs to someone else AND the resource is highly sensitive THEN the risk is high" are conditional and non-linear, and an ensemble classifier can learn such interactions from labeled access events and evaluate them in microseconds. The central idea of this thesis is to couple such a classifier directly to the cryptographic boundary of the application: ciphertext is preserved through database retrieval, a behavioral risk score is computed per request, and the amount of data that is decrypted — all of it, a masked portion of it, or none of it — is decided adaptively, at runtime, for every individual request.

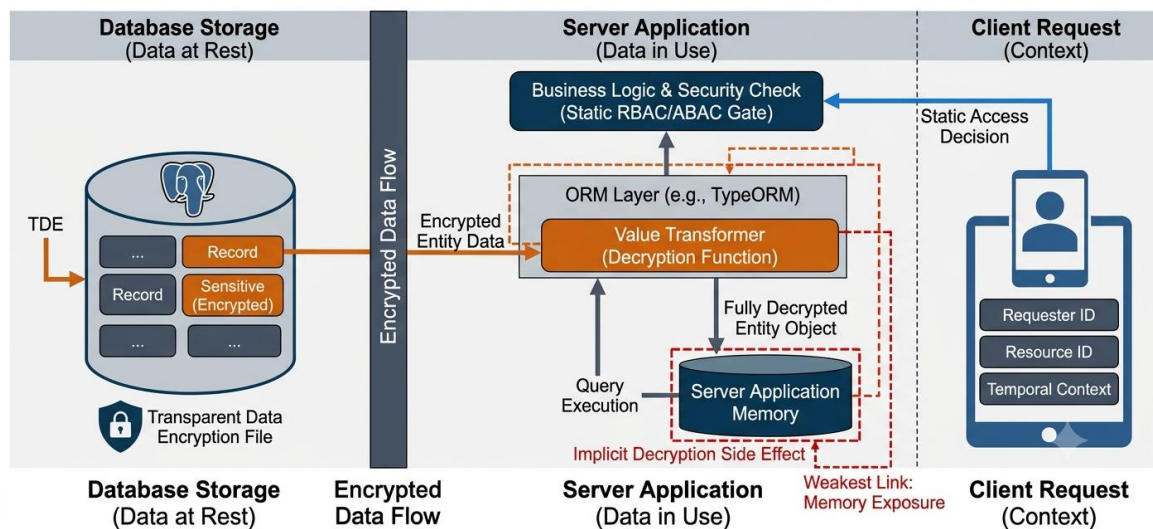


Figure 1.1: Baseline Architecture Showing Data-in-Use Weakness in Typical Enterprise Backends.

1.2 Problem Statement

The problem addressed in this thesis is the premature, unconditional decryption of encrypted database fields in application systems. In the standard ORM retrieval path, plaintext materializes in the server's heap memory before authorization logic beyond static role checks can execute. If the requester is executing an IDOR attack, an automated scraping script, or is operating a freshly compromised account, the sensitive plaintext is already exposed by the time any application-level check could throw an exception. Furthermore, because static access control is binary, systems have no graduated response available: they can only fully serve or fully reject a request, so borderline-suspicious behavior is either over-served (data leakage) or over-blocked (workflow disruption).

Formally, given an authenticated request for an encrypted resource, the system must decide — before any decryption occurs — a risk class $c \in \{\text{LOW}, \text{MEDIUM}, \text{HIGH}\}$ from a runtime feature vector x describing the requester's role, the resource's sensitivity, the temporal context, record ownership, request velocity, and recent authentication failures, and must then enforce the corresponding cryptographic policy: full decryption, partial decryption with masking, or denial in which zero application-level decryption calls are executed. This thesis investigates whether such a decision pipeline can be made simultaneously accurate (high attack interception, low false alarms), fast (negligible per-request latency), and architecturally clean inside a production-grade backend framework.

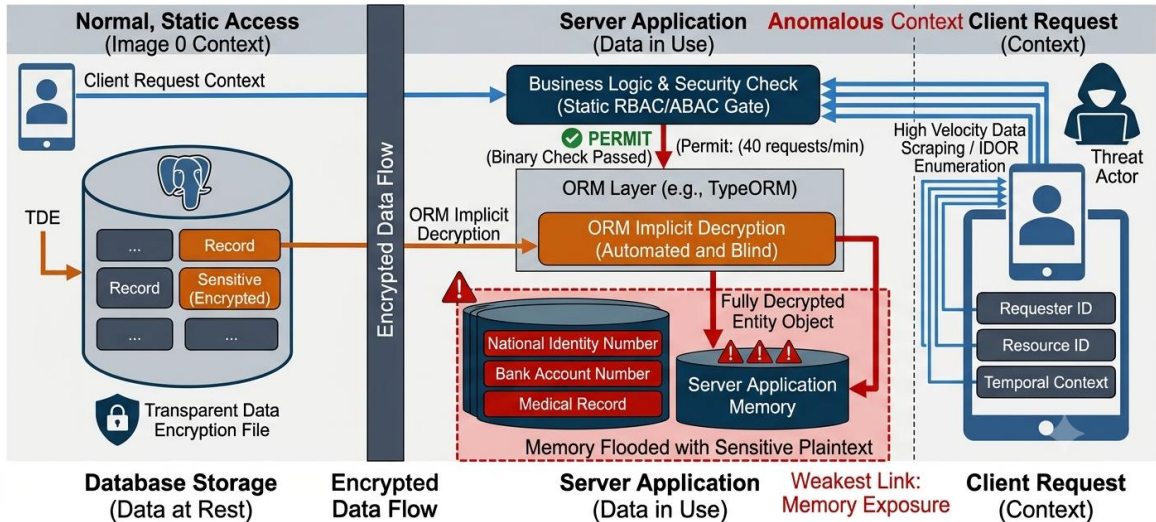


Figure 1.2: Analysis of Anomalous Context Attacks Exploiting Static Access Controls.

1.3 Research Gap

Industry systems widely deploy three of the four required layers: TDE and storage encryption for data at rest, application-level field encryption for data before storage, and static RBAC/ABAC for endpoint authorization. Zero Trust architectures come philosophically closest to the missing layer — they advocate continuous, risk-based verification — but existing implementations govern authentication and network access, not decryption behavior or data-exposure granularity. What is largely absent, both in industry practice and in the reviewed literature (Chapter II), is a layer that adaptively controls decryption and exposure at runtime, per request, as a function of behavioral risk: ML-driven decryption policies, adaptive response masking, runtime encryption/decryption decision logic, and per-request security behavior. This four-element combination, enforced at the exact boundary where ciphertext becomes plaintext, is the specific gap this thesis addresses.

1.4 Research Objectives

The aim of this thesis is to design, implement, and evaluate an adaptive, machine learning-driven data access and decryption control framework in which the exposure of encrypted application data is decided per request from runtime behavioral evidence — so that legitimate users experience full, uninterrupted access while behaviorally anomalous requests receive progressively masked data or are denied before any plaintext is produced.

The objectives of this research are as follows:

1. To engineer a reproducible, domain-rule-grounded synthetic dataset of labeled access events that realistically models legitimate workflows and attack behaviors (IDOR, scraping, credential stuffing) across three risk tiers.
2. To select, train, and tune a classifier that maps a runtime behavioral feature vector to a three-tier risk level with publication-grade accuracy, using a linear baseline for comparison and a quarantined test set for unbiased evaluation.
3. To transpile the trained model into dependency-free JavaScript executing natively inside the Node.js runtime, eliminating inference services and network hops from the request path.

4. To re-engineer the ORM retrieval path of a NestJS/PostgreSQL backend into a late-binding cryptographic design in which ciphertext survives entity hydration and decryption becomes an explicit, policy-gated action.
5. To integrate feature extraction, in-process inference, and a graduated policy engine (full decryption / partial masking / denial) into the backend, and to verify the complete pipeline end-to-end, including the property that the denial path performs zero decryption.

1.5 Research Questions

- **RQ1:** Can a small set of server-controlled behavioral features (role, sensitivity, time, ownership, velocity, authentication failures) discriminate legitimate access from attack behavior accurately enough to gate decryption?
- **RQ2:** Can the trained model be executed inside the application process with latency small enough to be negligible on every sensitive read?
- **RQ3:** Does a graduated, three-tier cryptographic policy reduce data exposure under attack while preserving workflow continuity for legitimate and borderline users?

1.6 Significance of the Study

Methodologically, the thesis contributes the late-binding cryptography pattern (deferring ORM decryption to an application-level gatekeeper), a strict train-time/runtime feature-mapping discipline that makes model transpilation sound, and a graduated risk-to-policy design that introduces a middle response between "serve everything" and "block everything." Practically, the framework applies directly to domains holding high-value field-encrypted data — banking and fintech, healthcare, e-commerce, government systems - where insider misuse, account takeover, and bulk extraction are primary threats. From a systems perspective, the work demonstrates that this protection can be obtained without new infrastructure: the transpiled model adds approximately fifty microseconds of in-process computation per request, and the entire adaptive layer is packaged as a reusable backend module.

1.7 Scope of the Study

The study covers: the design and generation of a synthetic access-event corpus; training, tuning, and evaluation of the risk classifier; transpilation of the model to JavaScript; the late-binding redesign of application-level encryption; runtime feature extraction from PostgreSQL-backed audit tables; the graduated policy engine with field-level masking; and offline plus end-to-end experimental evaluation. The prototype protects a representative high-sensitivity domain aggregate (bank account details with seven encrypted fields) inside a NestJS modular monolith. Transparent Data Encryption is treated as a deployment-level prerequisite of the layered architecture rather than implemented in application code; live production traffic, multi-node deployment, and field trials with real users are outside the scope of this undergraduate thesis.

1.8 Thesis Organization

Chapter II reviews the problem background, the role of machine learning in adaptive security, and related studies, and derives the research gap. Chapter III presents the research methodology: the layered security architecture, dataset generation policy, feature engineering, model development, transpilation technique, backend integration, and evaluation design. Chapter IV reports the experimental studies: environment, parameters, baselines, offline classification results, transpilation fidelity, and end-to-end adaptive behavior. Chapter V discusses engineering, ethical, and professional considerations. Chapter VI maps the work to complex engineering problems and activities. Chapter VII concludes with the major findings, limitations, and future recommendations. The report closes with references and follows the thesis plan shown in Figure 1.1.

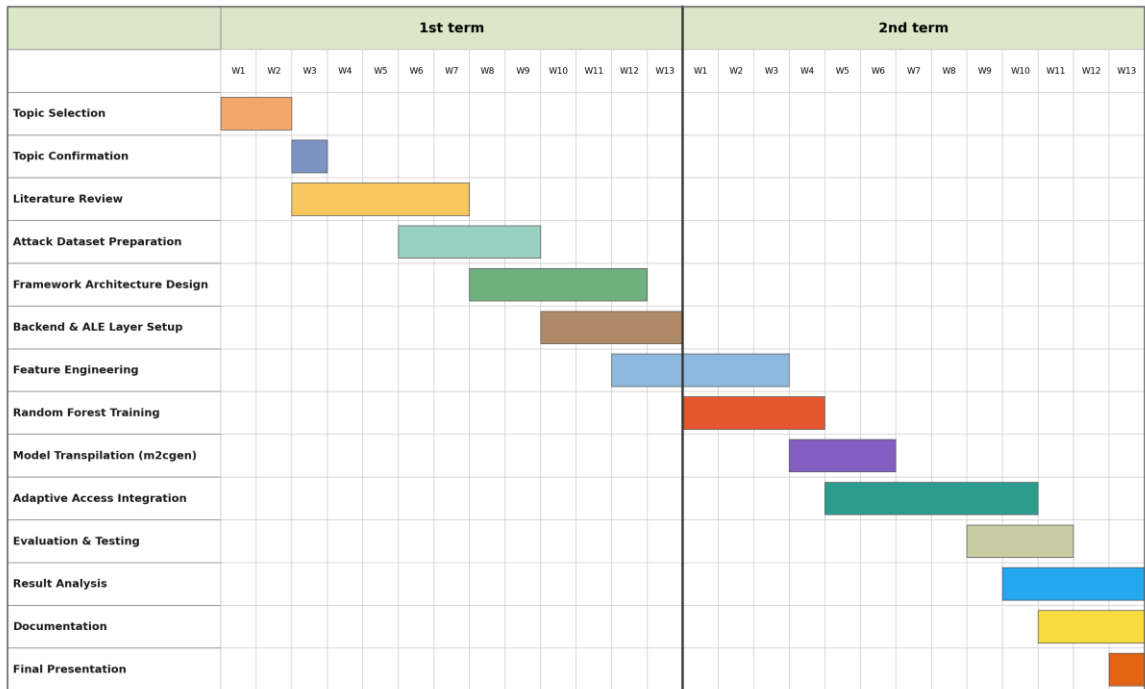


Figure 1.3: Gantt chart showing the week-by-week project plan across the two academic terms of the thesis.

CHAPTER II

Literature Review

Adaptive decryption control draws on cryptographic data protection, access-control theory, and machine learning-based security analytics. This chapter establishes the technical background of the problem, explains why machine learning is the appropriate instrument for runtime risk assessment, discusses the data required for such models, reviews prior work in the three contributing research streams, and derives the specific, verifiable gap that motivates the proposed framework.

2.1 Background of the Problem

Data protection mechanisms are conventionally organized by the state of the data they protect — data at rest, data in transit, and data in use — as illustrated in Figure 2.1. Of the three, the first two are served by mature, widely deployed controls, while the third remains the least defended and, increasingly, the most consequential: data in use is the state in which information exists as plaintext in server memory, and therefore the state in which a single unwarranted decryption converts every upstream protection into a formality. Protecting data in use is thus not an incremental hardening step but the decisive one — it is the last point at which exposure can still be prevented rather than merely detected, and it is precisely where conventional mechanisms fall silent.

Transparent Data Encryption (TDE) encrypts database files, backups, and storage artifacts; it is "transparent" precisely because the database engine decrypts pages automatically for any authenticated connection — which is also its limitation, since it offers no protection once the application queries the data [1], [2]. Application-Level Encryption (ALE) moves the cryptographic boundary into the application: selected sensitive fields are encrypted before storage, so the database, its administrators, its logs, and any SQL-injection read primitive observe only ciphertext, while key custody remains with the application [3], [14]. The authenticated cipher AES-256-GCM is the de facto standard for such field encryption: it provides confidentiality and integrity simultaneously, requires a unique initialization vector per encryption, and rejects tampered ciphertext through its authentication tag [4], [15]. Yet even the strongest field encryption shares TDE's blind spot in the data-in-use state: the moment an authenticated request triggers a read, the application decrypts the field unconditionally, and cryptography that faithfully guarded the

data for its entire stored lifetime is undone in microseconds by a single routine query. The strength of the cipher is irrelevant if the decision to invoke its decryption function is never examined — which is why the point of use, not the point of storage, is where protection ultimately succeeds or fails.

Access control complements encryption but does not close this gap. RBAC grants permissions through roles [3], [16]; ABAC generalizes this to policies over attributes of the subject, object, and environment [13], [17]. Both, however, are evaluated against static facts and produce binary verdicts: they decide once, at the gate, whether a principal may invoke an operation, and say nothing about how much plaintext that operation should subsequently materialize. The Zero Trust security model [9], [18] challenges the assumption that authenticated equals trustworthy and advocates continuous, context-aware verification — implicitly acknowledging that trust must be re-evaluated at the moment of use — but its realizations concentrate on authentication, network segmentation, and session management rather than on the decryption of stored data. The data-in-use boundary, where ciphertext becomes plaintext, is left ungoverned by every layer.

The practical importance of protecting this boundary is demonstrated by the threat classes motivating this thesis, all of which defeat static controls while exhibiting anomalous runtime behavior: IDOR attacks, in which an authenticated user manipulates object references to read other users' records [19]; high-velocity scraping and bulk exfiltration by valid sessions; credential stuffing, in which stolen credential lists eventually yield valid logins preceded by bursts of failures [19]; and insider misuse by fully authorized personnel. Each attacker holds valid credentials, so encryption at rest and access control at the gate are both satisfied — the only state in which these attacks can still be stopped is data in use. In each case, a static pipeline decrypts the requested data before any anomaly can be acted upon; detection, if it happens at all, happens after exposure, when the plaintext has already left the server. A protection mechanism that operates at the data-in-use boundary inverts this outcome: by conditioning decryption itself on runtime behavior, it makes prevention — not post-hoc detection — the default response, and it is this inversion that the framework proposed in this thesis realizes.

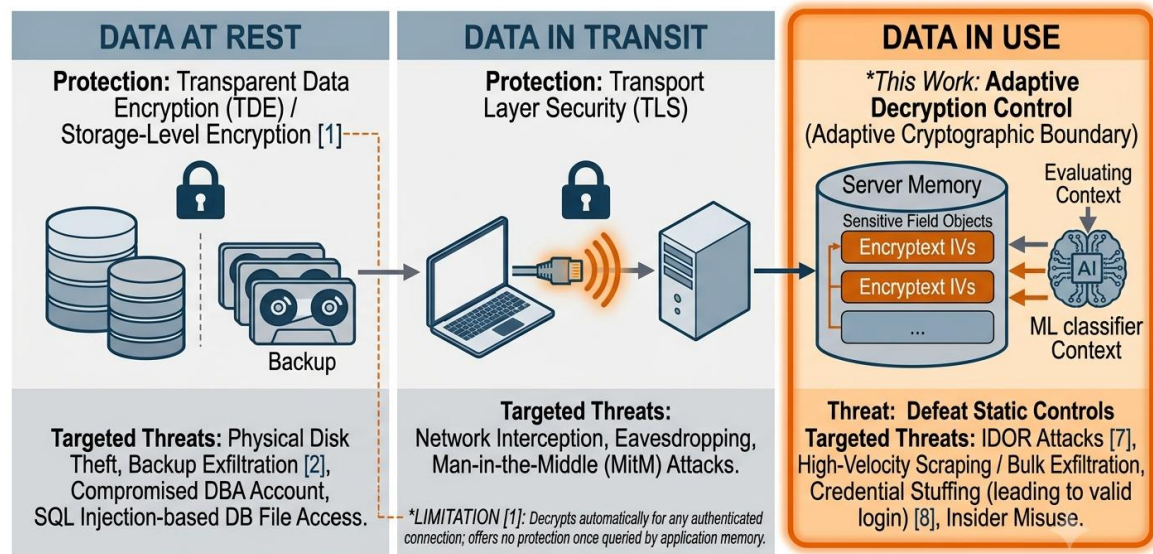


Figure 2.1: The three states of data and their protection mechanisms; the data-in-use state addressed by this thesis is highlighted.

2.2 Significance of Machine Learning in Adaptive Access Control

2.2.1 Machine Learning Basics

Supervised classification learns a function from labeled examples: given feature vectors x with known class labels y , a learning algorithm fits a model that predicts the label of unseen vectors. Model families differ in the decision boundaries they can represent: linear models such as Logistic Regression separate classes with hyperplanes, whereas tree ensembles such as the Random Forest [20] combine many decision trees — each a hierarchy of threshold tests — and aggregate their votes, naturally representing conditional, interacting rules. Disciplined evaluation requires separating data into training, validation (for hyperparameter tuning), and test partitions, with the test partition quarantined until a single final measurement.

2.2.2 Machine Learning for Automatic Risk Detection

Security risk assessment is a natural classification problem: each access request is an event described by measurable features, and the desired output is a risk tier that a policy engine can act upon [6], [7], as illustrated in Figure 2.2. Crucially, the rules that human security analysts write — "a customer touching another customer's banking record is dangerous; a moderator doing the same on low-sensitivity data is routine" — are conjunctions across

several attributes. Such role-asymmetric, conditional logic is exactly what tree ensembles capture and what linear models cannot, a distinction verified empirically in Chapter IV.

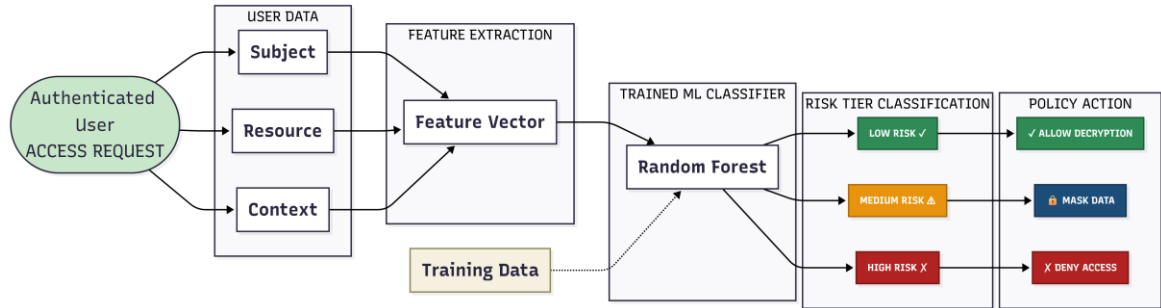


Figure 2.2: Concept of machine learning-based automatic risk assessment of access requests.

2.2.3 Benefits of ML-Based Risk Assessment

Relative to hand-maintained rule engines, a learned risk model offers: representation of non-linear feature interactions without enumerating every rule combination; calibrated class probabilities that support graduated (rather than binary) policies; microsecond-scale inference when the model is compiled into native code paths; consistency, in that identical evidence always yields identical decisions; and auditability, since ensemble models expose per-feature importance rankings that can be compared against the threat model for validation.

2.2.4 Data Requirements for Training ML Models

A reliable classifier requires labeled examples that cover the behavior space: thousands of events spanning all roles, sensitivity tiers, temporal contexts, and — critically — the attack patterns of interest, with realistic class imbalance (attacks are rare relative to legitimate traffic). Labels must encode defensible domain semantics, and a controlled level of label noise is desirable so that the model generalizes rather than memorizes. Because real labeled corpora of per-request decryption decisions do not publicly exist, and production security logs of this granularity cannot ethically be published, these requirements are met in this thesis through principled synthetic data generation (Section 2.3, Chapter III).

2.3 Benchmark Dataset and Procedure of Data Collection

No public benchmark exists for field-granular, application-layer decryption decisions. Established intrusion-detection corpora (e.g., network-flow datasets) describe packet- and flow-level phenomena, not the application-level question "how much of this encrypted record should this authenticated user see right now?". Publishing real banking access logs at this granularity would itself constitute a privacy breach. Consequently, this thesis constructs a synthetic corpus of 10,000 access events whose feature distributions (role mix, sensitivity mix, office-hours ratio, Poisson-distributed request velocity with injected scraping bursts, exponentially decaying failed-login counts) and whose labeling rules are grounded in the documented behavior of the threat classes of Section 2.1. The generator is fully deterministic under a fixed random seed, making the corpus bit-for-bit reproducible; its complete generation policy is a methodological contribution described in Section 3.2.

2.4 Review of Previous Studies

2.4.1 Studies on Cryptographic Protection of Stored Data

A substantial literature covers encryption of data at rest [10], [12]: database-engine TDE and its key-management practice [1], [2], field-level and searchable encryption schemes that trade functionality against leakage [2], and architectural guidance for application-managed keys [3], [11]. Advanced cryptographic directions — fully homomorphic encryption and secure enclaves — aim to compute on protected data, but their cost and complexity keep them far from routine web-backend deployment [4]. Across this stream, the retrieval policy is uniformly static: once a principal is authorized, decryption is unconditional. None of these work conditions the act of decryption on runtime behavior.

2.4.2 Studies on Access Control Models

RBAC [3] and ABAC [3], [5] are the deployed workhorses of authorization. Risk-based and risk-adaptive access control (RAdAC) research extends them by introducing a quantified risk estimate into the authorization decision [5]; studies in this stream propose risk formulas over user trustworthiness, resource value, and context, and several incorporate learning components for scoring [5]. However, the enforced decision remains admission-shaped — allow or deny an operation — and the enforcement point is the policy engine, not the cryptographic boundary. The graduated middle response (partial

decryption with masking) and the guarantee that denial precedes any plaintext materialization are absent.

2.4.3 Studies on ML-Based Security Analytics

Machine learning is well established in intrusion and anomaly detection: conventional classifiers (decision trees, random forests, support vector machines) over network and host telemetry [4], [5], deep models over sequential logs, and user/entity behavior analytics for insider-threat detection. Two recurring properties limit their applicability to the present problem. First, they are predominantly detective: they raise alerts for analysts after the fact, rather than gating a security-critical action inline. Second, their deployment pattern (separate inference services, streaming pipelines) adds latency and infrastructure that are difficult to justify on every database read of a web request. In-process inference over transpiled models addresses the second property but has not, in the reviewed literature, been coupled to decryption control.

2.4.4 Studies on Zero Trust and Data-Centric Security

Zero Trust architecture guidance [5] and continuous-authentication research operationalize the principle of ongoing verification, and dynamic data masking is available in commercial database products as a presentation-layer control [1]. Yet masking policies in such products are statically configured per role or column, not driven by per-request behavioral risk; and continuous-authentication systems act on sessions (forcing re-authentication or termination), not on the granularity of decrypted data. The reviewed streams thus contribute all the necessary ingredients — strong field encryption, risk quantification, learned behavioral scoring, and masking primitives — without combining them at the point where ciphertext becomes plaintext.

2.5 Summary of Existing Methods

Table 2.1 summarizes the reviewed approach families against the four properties required for data-in-use protection.

Table 2.1: Summary of existing approach families and the properties required for adaptive data-in-use protection.

Approach family	Protects data in use	Behavioral / adaptive	Graduated exposure	Acts before plaintext exists
TDE / storage encryption [1], [2]	No	No	No	—
Static ALE (field encryption) [3]	No (decrypts on read)	No	No	No
RBAC / ABAC [3], [5]	Partially (binary gate)	No	No	Yes, but static
Risk-adaptive access control [5]	Partially	Yes	No (allow/deny)	Policy layer only
ML intrusion / anomaly detection [4], [5]	No (detective)	Yes	No	No (post-hoc alerts)
Dynamic data masking [1]	Presentation layer	No (static rules)	Partially	No
This thesis	Yes	Yes (ML, per request)	Yes (decrypt / mask / deny)	Yes (deny path decrypts nothing)

2.6 Identified Research Gap

Four weaknesses recur across the reviewed literature. First, detection after exposure: analytics systems observe and alert, but by the time an alert exists the plaintext has already been served. Second, binary decisions: access-control research, even when risk-aware, retains an allow/deny vocabulary and therefore cannot serve borderline behavior safely. Third, decoupling from cryptography: no reviewed system makes the decryption operation itself the enforcement point; encryption research assumes static retrieval, and access-control research stops at the policy engine. Fourth, deployment friction: learned models are typically served from separate infrastructure whose latency and operational cost are disproportionate to a per-read decision. Any framework closing the data-in-use gap must therefore be preventive, graduated, cryptographically coupled, and cheap enough to run on every read.

No reviewed work combines: (i) preservation of ciphertext through ORM retrieval so that decryption is an explicit, deferrable action; (ii) a learned, per-request behavioral risk score computed from server-controlled evidence; (iii) a graduated three-tier cryptographic policy in which the middle tier partially decrypts and masks; and (iv) in-process, microsecond-scale inference requiring no additional infrastructure. This four-way integration — evaluated offline on a reproducible labeled corpus and end-to-end inside a production-grade backend — is the specific, verifiable gap addressed by this thesis.

2.7 Conceptual Framework and Proposed Research Direction

The proposed framework treats every sensitive read as a closed control loop: the request contributes to server-side behavioral logs; the logs and request context are distilled into a feature vector; a learned model converts the vector into a risk tier; the tier selects a cryptographic policy applied to the still-encrypted entity; and the decision itself is audited, feeding both security monitoring and future retraining. Two design commitments follow from the critical analysis: the enforcement point is the decryption boundary (late-binding cryptography), and the model executes inside the application process (transpiled inference). Chapter III develops this framework into a complete methodology.

2.8 Chapter Summary

Encryption research protects stored bytes but decrypts unconditionally on read; access-control research adapts to risk but stops at binary admission decisions far from the cryptographic boundary; ML security analytics detects anomalies but after exposure and with heavyweight serving infrastructure. The gap — adaptive, graduated, ML-driven control of decryption itself — defines the methodology presented next.

CHAPTER III

Research Methodology

Transforming the data-in-use problem into a working adaptive framework requires a layered security architecture, a principled synthetic dataset, a disciplined feature-engineering scheme shared between training and runtime, a carefully selected and tuned classifier, a zero-dependency transpilation technique, and a backend integration in which decryption is gated by the model's verdict. This chapter presents each component and the validation strategy and metrics used to evaluate them.

3.1 Research Framework

The research framework is a four-stage pipeline (Figure 3.1): (1) synthetic security dataset engineering, in which 10,000 labeled access events are generated under explicit domain rules; (2) model development, in which a Random Forest classifier is trained, tuned against a linear baseline, and evaluated on a quarantined test partition; (3) zero-dependency transpilation, in which the trained model is exported as pure JavaScript for in-process execution; and (4) backend integration and adaptive enforcement, in which the model gates decryption inside a NestJS/PostgreSQL application through a graduated policy engine.

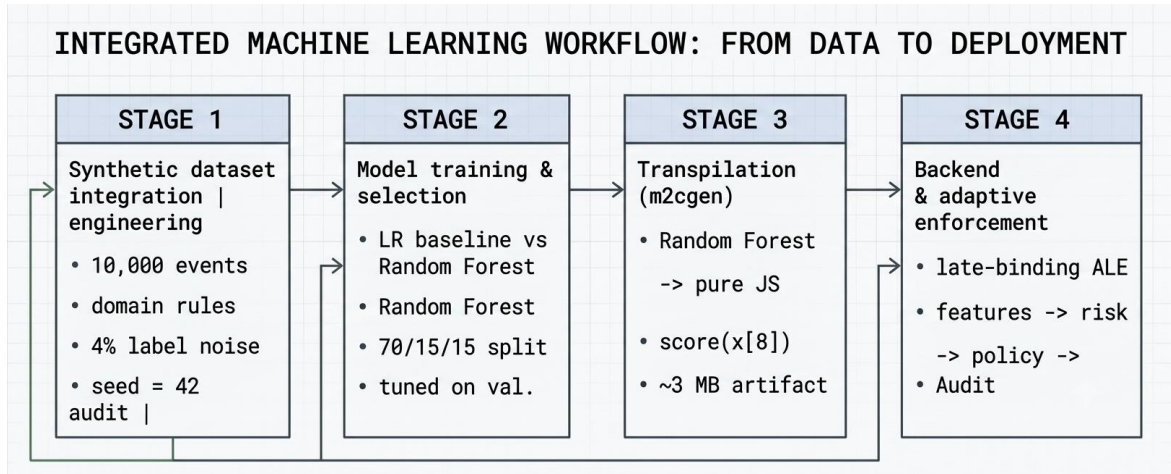


Figure 3.1: Four-stage research framework of the thesis, from dataset engineering to adaptive enforcement in the backend.

At runtime the framework is realized as an eight-layer security architecture (Figure 3.2). Layers 1–3 are the conventional application tiers (client, JWT authentication with static

RBAC, business logic with access logging). Layer 4 — the adaptive machine learning decryption control layer — is the core contribution of this thesis. Layers 5–7 are the cryptographic and storage tiers (application-level AES-256-GCM field encryption operating in late-binding mode, the database access layer, and TDE-protected storage as a deployment prerequisite). Layer 8 is the audit and feedback tier whose tables simultaneously serve as security telemetry and as the source of the behavioral features consumed by Layer 4.

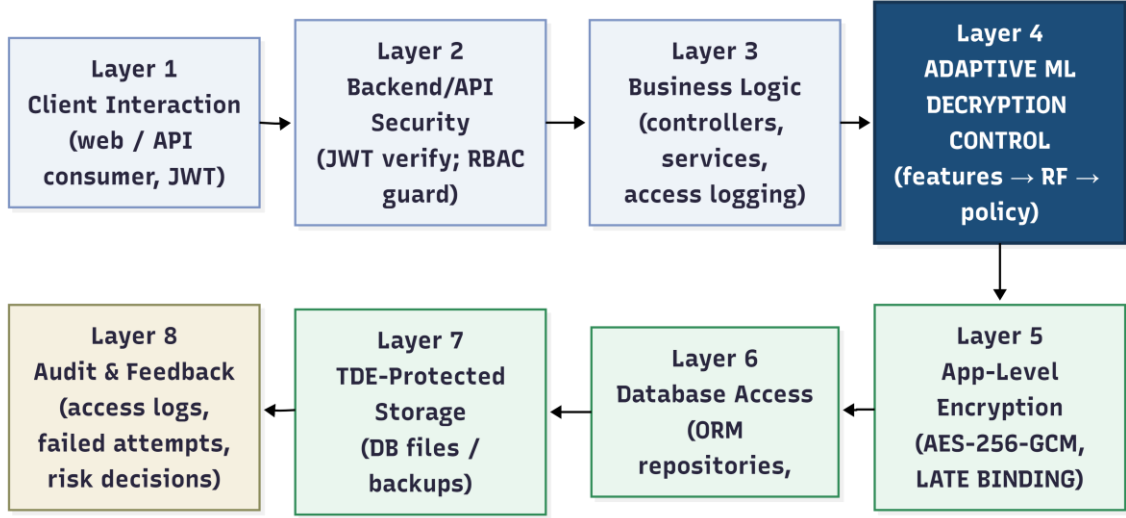


Figure 3.2: Eight-layer security architecture of the proposed framework; Layer 4 is the thesis contribution.

Problem formulation. Each authenticated request for an encrypted resource is described by a feature vector x of eight numeric elements derived from six business features (Section 3.3). A classifier maps x to a probability vector $P(c | x)$ over risk classes $c \in \{0 \text{ (LOW)}, 1 \text{ (MEDIUM)}, 2 \text{ (HIGH)}\}$; the predicted class is the argmax . A deterministic policy function then maps the class to a cryptographic action: FULL_DECRYPT, PARTIAL_MASK, or ACCESS_DENIED. The framework is required to satisfy two structural invariants: ciphertext must survive database retrieval unmodified (late binding), and the ACCESS_DENIED path must execute zero decryption operations.

3.2 Data Collection and Synthetic Dataset Generation

Because no public corpus of application-layer decryption decisions exists (Section 2.3), the training data is synthesized under an explicit, reproducible generative policy. The generator produces 10,000 access events under a fixed random seed (42) in three steps:

sampling of feature marginal distributions, labeling by role-aware non-linear business rules, and injection of controlled stochastic label noise.

Step 1 — feature distributions. Table 3.1 lists the marginal distribution of each feature. Two distributions deserve emphasis. The request-velocity feature follows a Poisson distribution with mean 4 (modeling organic human browsing of 1–6 requests per minute), after which a randomly chosen 5% of rows are overwritten with uniform values between 15 and 64 — injecting the bimodal scraping bursts characteristic of automated exfiltration. The failed-attempt feature follows a discrete exponential-decay distribution in which 88% of events have zero preceding failures and a long tail reaches eight, modeling brute-force campaigns. Record ownership is role-conditional: customers access their own records 95% of the time (the 5% complement models IDOR attempts), whereas moderators and administrators routinely access records owned by others (15% match) — teaching the model that the same feature value carries different meaning at different privilege levels.

Table 3.1: Marginal feature distributions of the synthetic access-event corpus (10,000 samples, seed 42).

Feature	Distribution	Parameters / rationale
user_role	Categorical	customer 75%, moderator 20%, admin 5% — read traffic is customer-dominated
resource_sensitivity	Categorical	LOW 60%, MEDIUM 30%, HIGH 10% — most reads touch low/medium-tier fields
is_office_hours	Bernoulli	$P(1) = 0.80$ — operational-hours dominance
record_owner_match	Role-conditional Bernoulli	customers: $P(\text{own record}) = 0.95$; staff roles: $P(\text{match}) = 0.15$
recent_request_count	Poisson($\lambda=4$) + injected attack mass	5% of rows overwritten with Uniform{15...64} scraping bursts
failed_attempt_count	Discrete exponential decay over {0...8}	$P(0) = 0.88$; long tail models brute-force campaigns

Step 2 — labeling rules. Labels encode a security analyst’s judgment as conjunctions over 2–4 features; HIGH-risk rules (Table 3.2) dominate MEDIUM-risk rules (Table 3.3), and all remaining traffic is LOW. The rule system is deliberately deep and role-asymmetric, which forces any successful model to learn feature interactions rather than single thresholds.

Table 3.2: HIGH-risk labeling rules of the dataset generator.

HIGH-risk rule (label = 2)	Threat interpretation
customer $\wedge$ owner_match = 0 $\wedge$ sensitivity $\in$ {MEDIUM, HIGH}	IDOR on protected data
customer $\wedge$ request_count > 15	Scraping / bulk exfiltration velocity
moderator $\wedge$ sensitivity = HIGH	Support tier has no business on HIGH-tier data
moderator $\wedge$ request_count > 25 $\wedge$ owner_match = 0	Bulk cross-record harvesting
admin $\wedge$ off-hours $\wedge$ sensitivity = HIGH $\wedge$ request_count > 20	Compromised super-user pattern
failed_attempts $\geq$ 5 (any role)	Post-brute-force account takeover

Table 3.3: MEDIUM-risk labeling rules of the dataset generator.

MEDIUM-risk rule (label = 1)	Interpretation
customer $\wedge$ owner_match = 0 $\wedge$ sensitivity = LOW	Cross-record access on public-tier data
customer $\wedge$ request_count $\in$ [9, 15]	Elevated velocity below the attack threshold
moderator $\wedge$ off-hours $\wedge$ sensitivity = MEDIUM	Unusual time for support work on personal data
admin $\wedge$ failed_attempts $\in$ {3, 4}	Moderate authentication friction on a privileged account
(moderator $\vee$ admin) $\wedge$ request_count > 18	Staff-tier volume anomaly

Step 3 — controlled label noise. Four percent of rows receive a random label re-draw. A noise-free, rule-labeled corpus would allow a tree ensemble to memorize the rules to a trivially perfect score; the noise floor models real-world labeling ambiguity, caps achievable accuracy at a realistic ceiling, and produces a measurable generalization gap. Finally, a stratified 70/15/15 split yields the partitions of Table 3.4; the class imbalance is intentionally preserved (the deployment prior really is "most traffic is benign") and compensated in the learner rather than by resampling.

Table 3.4: Stratified dataset partitions and their roles.

Partition	Samples	Purpose	Class balance
Training	7,000 (70%)	Model fitting	Stratified: $\approx$ 85% LOW / $\approx$ 6% MEDIUM / $\approx$ 9% HIGH
Validation	1,500 (15%)	Hyperparameter tuning	Same (stratified)
Test	1,500 (15%)	Quarantined; used once for final metrics	1,274 LOW / 87 MEDIUM / 139 HIGH

3.3 Data Preprocessing and Feature Generation

Features were selected against five explicit criteria: attack discriminativeness (each feature must separate at least one threat class from legitimate behavior); runtime extractability from server-controlled sources only — the JWT identity, the server clock, and database state — so that a client cannot influence its own risk score except by actually behaving differently; low extraction latency; transpilation compatibility, meaning stateless numeric encodings only; and explainability, so that every feature carries a defensible security interpretation. Table 3.5 lists the six business features, their runtime sources, and rationale.

Table 3.5: The six business features: sources, types, and selection rationale.

Feature	Type	Runtime source	Security rationale
user_role	Categorical	Verified JWT claim	Privilege context; identical behavior means different things at different privilege levels
resource_sensitivity	Ordinal (LOW/MEDIUM/HIGH)	Static, developer-declared classification of the data domain	Data tier; never inferred from the request (a client could lie)
is_office_hours	Binary	Server clock: Mon–Fri, 09:00–17:00	Temporal anomaly; off-hours access to sensitive data is a classic compromise indicator
record_owner_match	Binary	JWT identity vs. the fetched row’s owner column	IDOR detection; both sides are server-controlled and unforgeable
recent_request_count	Integer	Count of access-log rows in the last 60 seconds (indexed query)	Velocity; the primary signature of automated scraping
failed_attempt_count	Integer	Count of failed-login rows in the last 24 hours (indexed query)	Authentication friction preceding access marks credential-stuffing account takeover

Two window choices are deliberate. The 60-second velocity window matches the training-data definition exactly, and the request being scored is included in its own count (the access-log row is inserted before scoring), so an attacker’s thirtieth request in a minute is evaluated as the thirtieth. The 24-hour failure window is intentionally long: a short

window would reset to zero at precisely the moment a credential-stuffing attacker finally succeeds and begins reading data — the moment the feature exists to catch.

Static numerical mapping. Stateful preprocessing objects (fitted encoders or scalers) cannot be exported to JavaScript. The six business features are therefore expanded by hand-written, stateless mappings into an invariant eight-element numeric vector (Table 3.6) used identically by the Python training code and the TypeScript runtime. This index mapping is a project invariant: any reordering silently corrupts every prediction, which is why it is verified by parity tests (Section 3.5).

Table 3.6: Static numerical mapping: the invariant 8-element feature vector shared by training and runtime.

Index	Model feature	Encoding
x[0]	role_customer	1 if role = customer, else 0
x[1]	role_moderator	1 if role = moderator, else 0
x[2]	role_admin	1 if role = admin, else 0
x[3]	resource_sensitivity	LOW = 0, MEDIUM = 1, HIGH = 2
x[4]	is_office_hours	0 / 1
x[5]	record_owner_match	0 / 1
x[6]	recent_request_count	integer ≥ 1
x[7]	failed_attempt_count	integer ≥ 0

3.4 Algorithm and Model Development

3.4.1 Model Selection Rationale

Candidate families were assessed against the feature-engineering criteria plus one hard system constraint: the model must transpile to dependency-free JavaScript and execute in microseconds. Unsupervised anomaly detectors were rejected because the problem is supervised with three semantically distinct policy classes. Logistic Regression was retained as the linear baseline; its weighted F1 plateaued at approximately 0.75–0.81 because the labeling logic consists of conjunctions that no single hyperplane over eight features can represent. Neural networks were rejected on transpilation, explainability, and data-scale grounds. The Random Forest was selected: it represents rule conjunctions natively as root-to-leaf paths, is robust to the injected label noise through bagging, emits class-probability estimates by vote averaging (enabling the graduated policy), exposes per-

feature importances for auditability, and transpiles to flat, branch-predictable conditional code.

3.4.2 Random Forest Configuration and Training Policy

A Random Forest is an ensemble of T decision trees, each trained on a bootstrap sample with randomized feature selection at each split; the ensemble prediction is the average of the per-tree class votes, $P(c | x) = (1/T) \sum \text{votes}_t(c | x)$, and the risk class is the argmax of this vector. Table 3.7 documents the selected configuration and its justification. All tuning decisions consumed only the validation partition; the test partition remained sealed until the single final evaluation of Chapter IV.

Table 3.7: Random Forest hyperparameters and their justification.

Hyperparameter	Value	Justification
n_estimators (T)	75	Accuracy saturates near 75 trees on validation; more trees inflate the transpiled artifact linearly for no measurable F1 gain
max_depth (d)	12	Tuned over {8, 12, 16, unlimited} on the validation set; 12 captures the deepest 4-antecedent rule conjunctions with margin, while unlimited depth overfits the 4% noise and explodes transpiled code size (300+ deep trees $\approx$ 15 MB of JavaScript)
class_weight	balanced	Re-weights the loss by inverse class frequency, protecting minority-class recall (MEDIUM $\approx$ 6%, HIGH $\approx$ 9%) under the preserved imbalance
random_state	42	Reproducibility

3.4.3 Zero-Dependency Transpilation Technique

The backend runs on Node.js while the model is trained in Python; conventional bridges (a Python inference microservice, a spawned interpreter process, or an embedded model runtime) add milliseconds to seconds of latency plus operational infrastructure to every sensitive read. This thesis instead transpiles the fitted forest with the model-to-code generator m2cgen: each tree becomes a nested cascade of threshold comparisons over

$x[0\dots7]$, each leaf yields a class-vote vector, and the 75 vote vectors are averaged — an exact, dependency-free reproduction of the library model’s probability computation as approximately 3 MB of plain JavaScript exposing a single function $\text{score}(x) \rightarrow [\text{P}(\text{LOW}), \text{P}(\text{MEDIUM}), \text{P}(\text{HIGH})]$. The artifact is loaded once at process start; after a one-time parse cost, each inference is a pure in-memory function call executing in roughly fifty microseconds (measured in Section 4.5). The static numerical mapping of Section 3.3 is the prerequisite that makes this technique sound: because preprocessing is stateless and definitionally identical on both sides, the transpiled trees observe exactly the distribution they were trained on. Notably, the very first split of the first transpiled tree tests $x[6] \leq 15.5$ — the transpiled artifact visibly encodes the dataset’s dominant scraping threshold, a structural confirmation that the intended domain logic was learned.

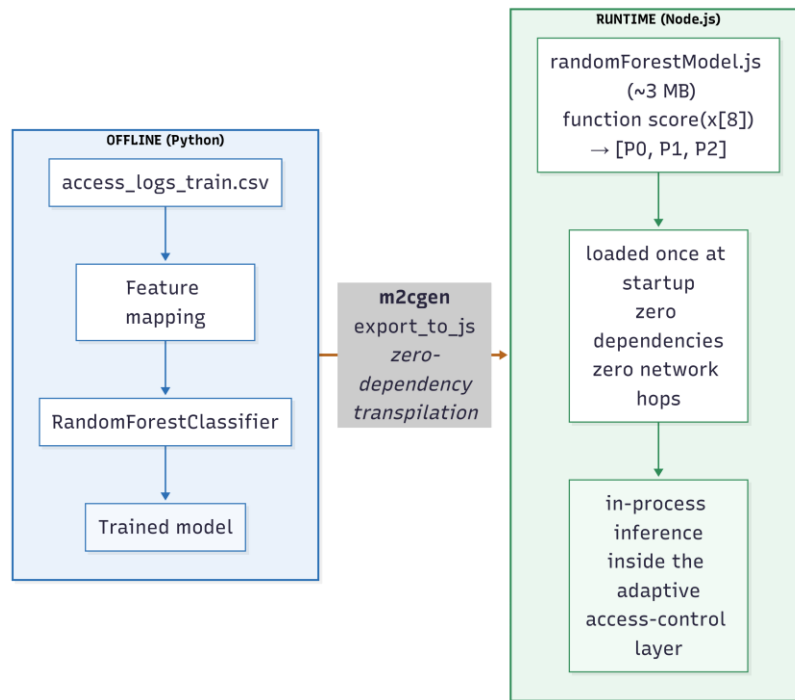


Figure 3.3: Offline training pipeline and zero-dependency transpilation into the backend runtime.

3.4.4 Late-Binding Application-Level Encryption

The pivotal architectural change is the redesign of the ORM value transformer. In the standard design, the transformer’s save-path function encrypts and its load-path function decrypts — meaning plaintext materializes during entity hydration, before any risk evaluation can run. In the late-binding design, the save path is unchanged (fields are encrypted with AES-256-GCM under a random 96-bit initialization vector, producing a versioned envelope of the form `version:IV:authTag:ciphertext`), but the load path returns

the raw ciphertext untouched. Three consequences follow: a repository query returns an entity whose sensitive properties are ciphertext strings, so retrieval itself performs no application-level decryption of any sensitive field; decryption becomes an explicit call that only one component — the adaptive gatekeeper — is permitted to make, after risk evaluation; and the decision ordering inverts from "decrypt, then (maybe) authorize" to "authorize adaptively, then (maybe, partially) decrypt." One schema detail supports the middle policy tier: the last four digits of the account number are stored as a separate plaintext column, enabling masked rendering of the account number with zero decryption.

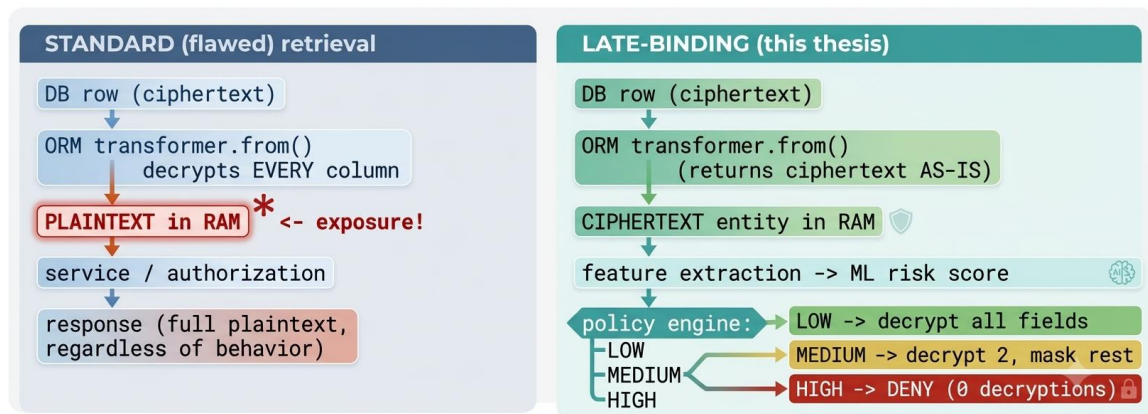


Figure 3.4: Standard ORM retrieval versus the late-binding cryptographic redesign.

3.4.5 Runtime Adaptive Components

Layer 4 comprises three cooperating components. The access context builder assembles the six business features for the current request: role from the verified JWT; sensitivity as the static classification declared by the domain service; office hours from the server clock; ownership by comparing the JWT identity against the owner column of the fetched row (never against any client-supplied parameter); and the two behavioral counts from indexed range queries over the audit tables (`access_requests`, populated by the controller before each protected read, and `failed_attempts`, populated by the authentication service on every failed password verification and refresh-token-reuse event). The two counts execute in parallel and cost one to two milliseconds combined.

The risk gatekeeper converts the context into the eight-element vector under the strict index mapping, invokes the transpiled score function, takes the argmax as the risk class, and writes an audit record containing the exact feature vector, the class probabilities, the decision, and the inference time — making every decision self-explaining and reviewable.

The policy engine enforces the mapping of Table III.8. Under FULL_DECRYPT all seven encrypted fields are decrypted. Under PARTIAL_MASK exactly two low-value fields are decrypted (branch name; account-holder name, which is then masked to initials); the account number is rendered from the stored plaintext last four digits without touching its ciphertext; and the four high-value identifiers remain hidden. Under ACCESS_DENIED an exception is thrown before any decryption call — the HIGH-risk path performs zero application-level decryption calls before returning denial. Figures 3.5 and 3.6 show the complete request flow and the decision mapping.

Table 3.8: Graduated risk-to-policy mapping enforced by the adaptive layer.

Risk class	Policy	Decryptions performed	Client receives
0 — LOW	FULL_DECRYPT	All 7 encrypted fields	Full plaintext plus a security metadata block (risk tier, decision, class probabilities)
1 — MEDIUM	PARTIAL_MASK	Exactly 2 fields (branch name; holder name, masked to initials)	Masked view: account number as ****<last-4> with zero decryption; routing / SWIFT / IBAN / identity hidden
2 — HIGH	ACCESS_DENIED	None — exception precedes any decryption	HTTP 403; the incident is recorded

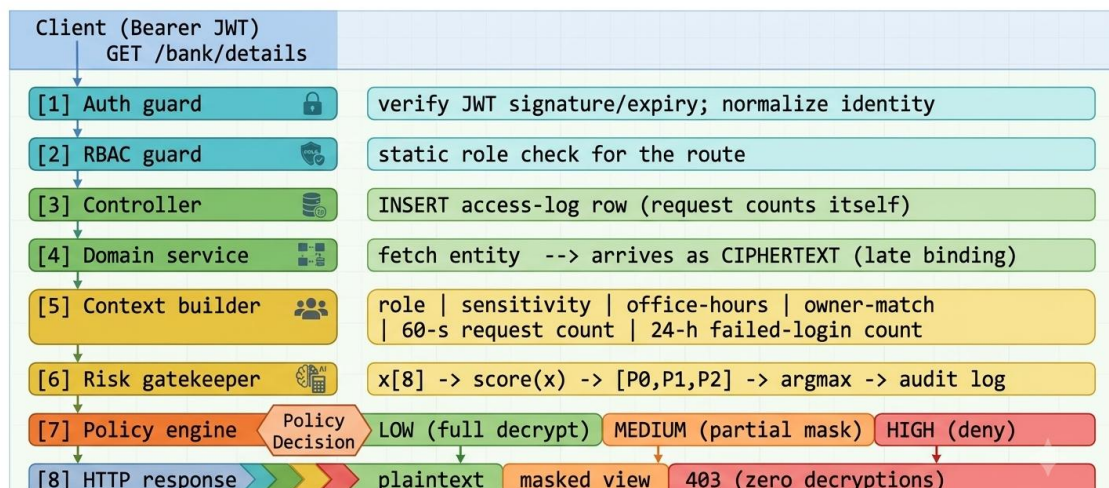


Figure 3.5: End-to-end runtime flow of one protected request through the adaptive framework.

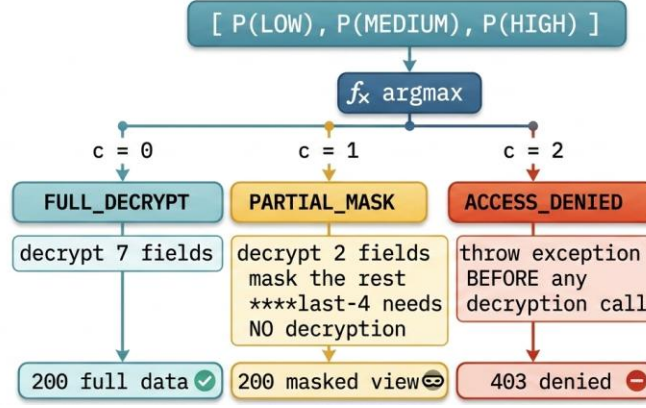


Figure 3.6: Risk-to-policy decision flow of the graduated enforcement engine.

3.5 Experimental Design and Validation Strategy

Three validation mechanisms address model quality, transpilation correctness, and system behavior respectively. (1) Offline discipline: the stratified 70/15/15 split, tuning restricted to the validation partition, and a quarantined test partition evaluated exactly once license the reported test metrics as an unbiased generalization estimate; a Logistic Regression baseline trained under identical conditions quantifies the value of modeling feature interactions. (2) Transpilation fidelity: known feature vectors with domain-expected outcomes are fed through the transpiled JavaScript artifact (not the Python model), verifying that the exported arithmetic reproduces the trained behavior, and the feature space is scanned to map the decision bands (for example, where the MEDIUM tier lies for a customer reading their own high-sensitivity record). (3) End-to-end verification: the compiled backend is exercised with genuine AES-256-GCM ciphertext entities under contexts engineered to produce each risk tier, confirming the three response shapes and the zero-decryption property of the denial path. All stochastic components are seeded (seed 42), making the entire pipeline deterministic and reproducible.

3.6 Evaluation Metrics

Classification quality is measured per class by precision $P = TP / (TP + FP)$, recall $R = TP / (TP + FN)$, and their harmonic mean $F1 = 2PR / (P + R)$, aggregated as macro (unweighted mean) and weighted (support-weighted mean) averages, together with the full 3×3 confusion matrix. Because raw accuracy is misleading under class imbalance, two security-operational readings of the confusion matrix are additionally reported: the legitimate pass rate (fraction of truly LOW requests receiving full decryption —

operational friction) and the attack interception rate (fraction of truly HIGH requests denied outright). System-level metrics are the inference latency of the transpiled model (cold start and warm steady state), the feature-extraction latency, and the field-level exposure per policy tier (how many of the seven encrypted fields are decrypted on each path). Model auditability is assessed by comparing the learned feature-importance ranking against the threat model.

3.7 Significance of the Proposed Method

No reviewed study integrates the four elements this framework combines: (i) late-binding cryptography, which preserves ciphertext through ORM retrieval and turns decryption into a policy decision; (ii) a learned per-request risk score computed from unforgeable, server-controlled behavioral evidence; (iii) a graduated three-tier cryptographic policy whose middle tier partially decrypts and masks — including masking the primary identifier with zero decryption; and (iv) in-process, microsecond-scale inference through model transpilation, requiring no additional serving infrastructure. Every claimed element corresponds one-to-one to a mechanism defined in this chapter and exercised experimentally in Chapter IV.

CHAPTER IV

Experimental Studies

The proposed framework was implemented as a NestJS/PostgreSQL application with a Python training pipeline and evaluated in two stages: offline classification quality on the quarantined test partition, and end-to-end verification of the integrated adaptive pipeline, including transpilation fidelity, decision-band mapping, latency, and the field-level exposure of each policy tier. This chapter reports the environment, parameters, baselines, procedure, results, analysis, and discussion.

4.1 Experimental Environment

Table 4.1 summarizes the implementation and evaluation environment. The backend is a modular monolith; the adaptive security layer is packaged as a self-contained, dependency-injected module so that any domain module serving encrypted resources can import it. The demonstration domain is a bank-details aggregate with seven AES-256-GCM-encrypted fields (branch name, account-holder name, account number, routing number, SWIFT code, IBAN, and national identity number).

Table 4.1: Implementation environment of the framework.

Component	Detail
Backend framework	NestJS 11 (TypeScript) modular monolith
Runtime	Node.js v22.12.0 (V8 engine)
Database / ORM	PostgreSQL with TypeORM 0.3.x
Cryptography	AES-256-GCM (Node.js crypto), 96-bit random IV, versioned ciphertext envelope
ML training	Python 3, scikit-learn (RandomForestClassifier, LogisticRegression)
Transpilation	m2cgen (model-to-code generator) → pure JavaScript
Operating system	Microsoft Windows 11 (x64)
Hardware	Intel Core i5-1135G7 (11th Gen, 2.40 GHz), 8 GB RAM
Input / output format	CSV (dataset), JSON (API responses)

4.2 Parameter Settings

Table 4.2: Parameter settings used throughout the reported experiments.

Category	Parameter	Value
Dataset	Samples / random seed	10,000 / 42
Dataset	Label noise	4% random re-draw
Dataset	Split	70% train / 15% validation / 15% test (stratified)
Model	Trees / max depth	75 / 12
Model	class_weight	balanced
Features	Velocity window	60 seconds (request counts itself)
Features	Failed-attempt window	24 hours
Features	Office hours	Monday–Friday, $09:00 \leq \text{hour} < 17:00$
Crypto	Cipher / IV / tag	AES-256-GCM / 12-byte random IV / 16-byte auth tag
Policy	Risk $\rightarrow$ action	LOW $\rightarrow$ full decrypt; MEDIUM $\rightarrow$ partial mask; HIGH $\rightarrow$ deny

4.3 Baseline and Comparative Methods

Two baselines position the results. The model baseline is Logistic Regression trained on the identical partitions and features, quantifying how much of the achieved quality is due to modeling non-linear feature interactions. The system baseline is the conventional static pipeline — standard ORM decryption plus RBAC only — against which the adaptive framework’s behavior under each threat class is compared qualitatively in Section 4.7; under the static pipeline every authenticated, role-authorized request receives full plaintext by construction.

4.4 Procedure of Experimentation

1. Generate the synthetic corpus (seed 42) and verify its class distribution and partition stratification.
2. Train the Logistic Regression baseline and the Random Forest on the training partition; tune tree depth on the validation partition.
3. Evaluate the selected model exactly once on the quarantined test partition: classification report, confusion matrix, feature importances.

4. Transpile the forest to JavaScript; measure artifact size, cold-start cost, and warm inference latency inside Node.js.
5. Verify transpilation fidelity by scoring domain-expected scenarios through the JavaScript artifact, and scan the feature space to map the MEDIUM decision band.
6. Integrate the artifact into the backend (late-binding transformer, context builder, gatekeeper, policy engine) and run end-to-end scenarios against genuine AES-256-GCM ciphertext entities, capturing the three response shapes and confirming zero decryptions on the denial path.

4.5 Experimental Results

4.5.1 Offline Classification Performance

Table 4.3 reports the per-class results on the quarantined test set (1,500 samples) and Table 4.4 the confusion matrix. The Random Forest achieves an overall accuracy of 0.9780 and a weighted F1-score of 0.9777. Read operationally: of 1,274 legitimate (LOW) requests, only 11 were flagged — 5 masked and 6 blocked — a 99.14% legitimate pass rate; of 139 attack (HIGH) vectors, 129 were denied outright and 1 was masked, a 92.8% outright interception rate; and no truly-MEDIUM request was hard-blocked, confirming that the graduated middle tier absorbs boundary cases without workflow disruption.

Table 4.3: Per-class classification report on the quarantined test set (single final evaluation).

Class (policy)	Precision	Recall	F1-score	Support
LOW (full decrypt)	0.9836	0.9914	0.9875	1,274
MEDIUM (partial mask)	0.9259	0.8621	0.8929	87
HIGH (access denied)	0.9556	0.9281	0.9416	139
Accuracy	—	—	0.9780	1,500
Macro average	0.9550	0.9272	0.9407	1,500
Weighted average	0.9777	0.9780	0.9777	1,500

Table 4.4: Confusion matrix on the test set: 0.86% false-alarm rate on legitimate traffic; 92.8% outright attack interception; zero hard blocks of MEDIUM-tier users.

	Predicted LOW	Predicted MEDIUM	Predicted HIGH
Actual LOW (1,274)	1,263	5	6
Actual MEDIUM (87)	12	75	0
Actual HIGH (139)	9	1	129

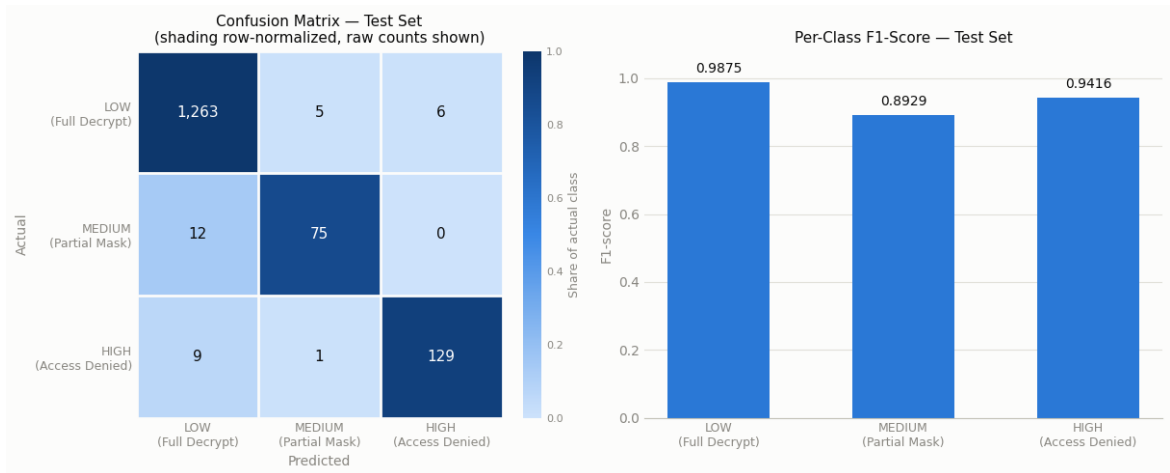


Figure 4.1: Visualization of the test-set confusion matrix (heatmap) and per-class F1-scores.

4.5.2 Feature Importance and Model Auditability

Table 4.5 lists the learned feature-importance ranking. The ranking independently reproduces the structure of the threat model without having been told it: the behavioral features (velocity, authentication friction, ownership) and the data-classification tier dominate, while static role and temporal context act as modifiers. This coherence between learned importances and domain reasoning is primary evidence that the model captured the intended security logic rather than dataset artifacts.

Table 4.5: Learned feature-importance ranking of the Random Forest.

Rank	Feature	Importance	Security interpretation
1	recent_request_count	0.4189	High-velocity scraping — the dominant learned signal
2	resource_sensitivity	0.1744	Data classification tier
3	failed_attempt_count	0.1376	Brute force / credential stuffing
4	record_owner_match	0.1292	IDOR / lateral movement
5	role_customer	0.0429	Base role contextual modifier
6	role_moderator	0.0378	Support-tier boundary modifier
7	is_office_hours	0.0358	Temporal anomaly weighting
8	role_admin	0.0233	Super-user baseline modifier

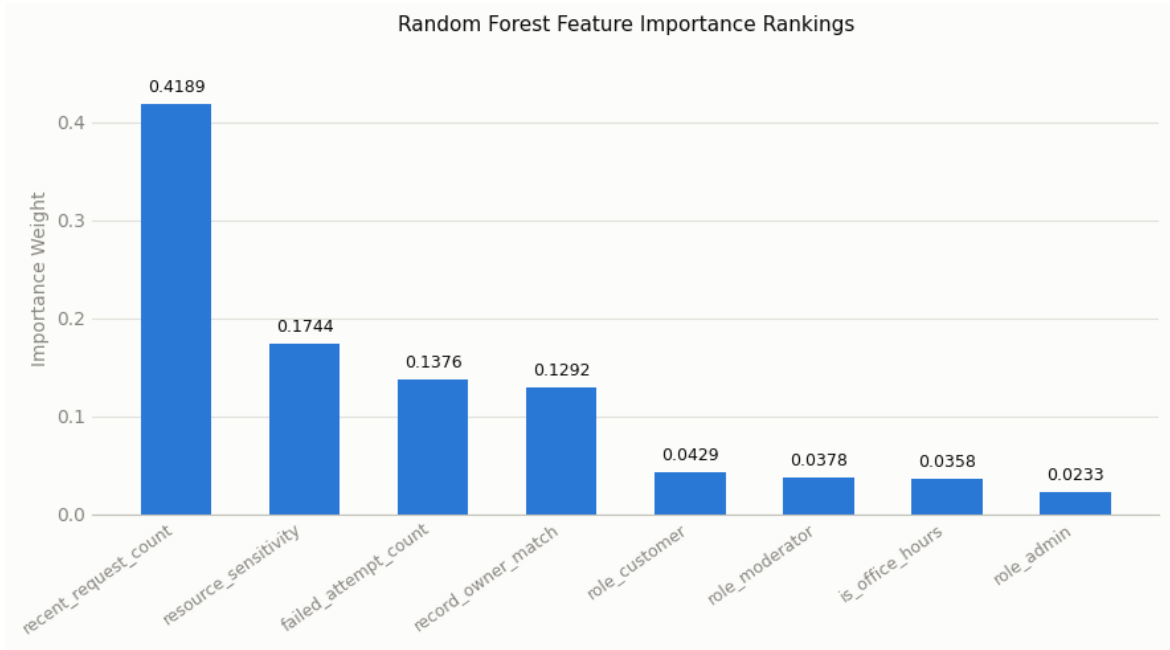


Figure 4.2: Feature-importance ranking of the trained model.

4.5.3 Transpiled Artifact: Fidelity and Latency

Table 4.6 reports the measured properties of the transpiled JavaScript artifact, and Table 4.7 the fidelity scenarios scored through the artifact inside Node.js. Every scenario produces the domain-expected verdict with decisive probability mass. A scan of the feature space for a customer reading their own HIGH-sensitivity record located the MEDIUM band at approximately 9–13 requests per minute (36 feature combinations in total), with the cleanest interior point — the ninth request within sixty seconds during office hours — classified MEDIUM at probability 0.982; beyond roughly 15 requests per minute the verdict flips to HIGH.

Table 4.6: Measured properties of the transpiled model inside the Node.js runtime.

Property	Measured value
Artifact size	≈ 3.05 MB of generated JavaScript (75 trees, depth ≤ 12)
Interface	$\text{score}(x[8]) \rightarrow [\text{P}(\text{LOW}), \text{P}(\text{MEDIUM}), \text{P}(\text{HIGH})]$
Cold start (first call: parse + JIT)	≈ 50 ms, once per process
Warm inference (steady state)	$\approx 49\text{--}50$ microseconds per call
Feature extraction (two indexed counts, parallel)	$\approx 1\text{--}2$ ms
Runtime dependencies	None (plain JavaScript arithmetic)

Table 4.7: Transpilation-fidelity scenarios scored through the JavaScript artifact.

Scenario (feature vector semantics)	Class probabilities [LOW, MED, HIGH]	Verdict	Expected
Legitimate customer, own record, office hours, 1 req/min, 0 failures	[0.987, 0.011, 0.002]	LOW	LOW
Same customer, 9th request within 60 s (office hours)	[0.018, 0.982, 0.000]	MEDIUM	MEDIUM
Scraper: 40 requests/min on HIGH-tier data	[0.000, 0.000, 1.000]	HIGH	HIGH
IDOR: customer, cross-record access, HIGH sensitivity	[0.010, 0.000, 0.990]	HIGH	HIGH
Credential stuffing: 8 recent failed logins, then access	[0.280, 0.000, 0.720]	HIGH	HIGH

4.5.4 End-to-End Adaptive Behavior

The integrated pipeline was exercised against entities holding genuine AES-256-GCM ciphertext under contexts engineered to produce each tier. Table 4.8 shows the field-level exposure of the same record under the three policies. On the LOW path all seven fields are decrypted and returned with a security metadata block. On the MEDIUM path exactly two decryption operations occur: the branch name, and the holder name which is masked to initials ("R**** A****"); the account number is rendered as "****6789" from the stored plaintext last-four column — its ciphertext is never decrypted — and the four high-value identifiers remain hidden. On the HIGH path the request is rejected with HTTP 403 and zero application-level decryption calls execute, so the framework never invokes its decryption routine for any field of the protected row. The observable consequence is the framework's defining property: the same endpoint, for the same user, returns full plaintext, a masked view, or a denial purely as a function of runtime behavior — full access at one to eight requests per minute, masking at roughly nine to thirteen, denial beyond fifteen or after five or more recent failed logins or on cross-record access to sensitive data.

Table 4.8: Field-level exposure of the same encrypted record under the three adaptive policies.

Field	LOW — FULL_DECRYPT	MEDIUM — PARTIAL_MASK	HIGH — ACCESS_DENIED
Branch name	Plaintext (decrypted)	Plaintext (decrypted)	— (HTTP 403;
Account holder name	Plaintext (decrypted)	"R**** A****" (decrypted, masked)	no field is
Account number	Plaintext (decrypted)	"*****6789" (zero decryption; stored last- 4)	decrypted or
Routing number	Plaintext (decrypted)	Hidden	returned;
SWIFT code	Plaintext (decrypted)	Hidden	incident is
IBAN	Plaintext (decrypted)	Hidden	recorded)
Identity number	Plaintext (decrypted)	Hidden	
Decryption operations	7	2	0

4.6 Experimental Analysis

Per-class behavior. The LOW class is nearly perfect (F1 0.9875), which matters most operationally because it dominates traffic. The MEDIUM class is hardest (F1 0.8929): it is both the smallest class (5.8% of data) and semantically a boundary band between normal and attack behavior; its errors are benign in direction — 12 MEDIUM events received full decryption and none was hard-blocked. The HIGH class errs on the safe side: its 9 false negatives are low-and-slow outliers whose feature signatures genuinely overlap legitimate traffic, the known residual risk class discussed in Section 7.3.

Decision-band structure. The feature-space scan shows the model learned a graduated velocity response for self-access — LOW up to about eight requests per minute, MEDIUM at nine to thirteen, HIGH beyond fifteen — while cross-record access to sensitive data and heavy authentication failure histories jump directly to HIGH regardless of velocity. This matches the labeling rules’ intent: velocity is a graduated signal, whereas IDOR and account takeover are categorical ones.

Ablation-style reading. Because the linear baseline shares the identical data, features, and protocol, the ≈ 17 -point weighted-F1 gap (Section 4.7) isolates the value of modeling feature interactions. Similarly, because the transpiled artifact reproduces the trained model’s decisions on all fidelity scenarios, the deployment step contributes no measurable accuracy loss while removing every network hop and external dependency from the inference path.

Security-structural properties. Two properties hold by construction and were confirmed at runtime: features are unforgeable (every model input originates from the JWT signature, the server clock, or server-side tables — a client can influence its risk score only by actually behaving differently), and denial precedes decryption (the HIGH path throws before any cryptographic call, so a correct classification yields prevention, not detection).

4.7 Comparative Analysis

Model comparison. Table 4.9 compares the selected model against the linear baseline under the identical protocol. The Random Forest’s advantage concentrates exactly where the security semantics are conditional — the minority MEDIUM and HIGH classes — confirming the non-linearity argument of Section 3.4.1.

Table 4.9: Selected model versus the linear baseline under an identical protocol.

Model	Weighted F1 (test)	Interpretation
Logistic Regression (baseline)	$\approx 0.75\text{--}0.81$	A linear hyperplane cannot represent the conjunction-shaped security rules ($\text{role} \wedge \text{ownership} \wedge \text{sensitivity} \wedge \text{velocity}$)
Random Forest (selected)	0.9777	Rule conjunctions are represented natively as root-to-leaf paths; ≈ 17 -point absolute improvement

System comparison. Table 4.10 contrasts outcomes per threat class against the static baseline pipeline (standard ORM decryption plus RBAC), in which every authenticated, role-authorized request receives full plaintext by construction.

Table 4.10: Threat-by-threat comparison of the static baseline pipeline and the proposed adaptive framework.

Threat scenario	Static pipeline (ORM + RBAC)	Proposed adaptive framework
Legitimate self-access	Full plaintext	Full plaintext (99.14% pass rate) + audit trail
Moderate over-querying (9–13 req/min)	Full plaintext	Masked view; workflow continues
Scraping (40 req/min)	Full plaintext on every request	Denied at $P(\text{HIGH}) = 1.000$; zero decryption
IDOR cross-record read	Full plaintext if role-authorized	Denied at $P(\text{HIGH}) = 0.990$; zero decryption
Post-credential-stuffing access	Full plaintext	Denied at $P(\text{HIGH}) = 0.720$; zero

		decryption
Added latency per request	—	≈ 2 ms total (audit insert + feature counts); inference ≈ 50 μ s

4.8 Discussion of Findings

The results answer the research questions affirmatively. RQ1: six server-controlled features suffice to separate the three tiers at 0.9777 weighted F1, and the learned importance ranking mirrors the threat model, indicating genuine domain logic rather than artifacts. RQ2: transpiled in-process inference costs ≈ 50 microseconds warm — three orders of magnitude below the network cost of an inference service — making per-read scoring effectively free; total security overhead (≈ 2 ms) is dominated by the audit insert and feature queries. RQ3: the graduated policy demonstrably reduces exposure under attack (zero decryptions on denial; two decryptions and zero-decryption masking of the primary identifier under suspicion) while keeping 99.14% of legitimate traffic untouched and never hard-blocking borderline (MEDIUM) users. Each stated objective of Section 1.5 maps to a delivered, verified artifact: the dataset (Objective 1, Section 3.2), the trained and tuned model (Objective 2, Sections 3.4, 4.5.1), the transpiled artifact (Objective 3, Section 4.5.3), the late-binding integration (Objective 4, Section 3.4.4), and the verified end-to-end pipeline (Objective 5, Section 4.5.4).

4.9 Risk of Rule-Learning Bias in Synthetic Evaluation

Because the training corpus is generated under explicit labeling rules (Section 3.2) and Random Forests represent rule-like conjunctions natively as root-to-leaf paths (Section 3.4.1), part of the reported classification quality necessarily reflects the model's ability to recover the synthetic labeling policy itself rather than evidence of generalization to real attacker behavior. The 4% label-noise floor, the injected distributional overlap, and the quarantined test protocol prevent trivial memorization of individual rows, but they cannot prevent the model from learning the structure of the rules, because the test partition is drawn from the same generative process as the training partition. High accuracy on such a corpus is therefore a necessary condition for the framework's viability, not a sufficient one.

Consequently, the offline results of Sections 4.5–4.7 should be read as proof-of-concept validation: they demonstrate that the complete pipeline — behavioral feature extraction,

in-process transpiled inference, and graduated cryptographic policy enforcement — functions correctly, and that a learned model can enforce a coherent, explainable security policy at negligible per-request latency. They do not, by themselves, establish real-world attack-detection performance. Establishing that requires the retraining path already designed into the architecture: the Layer-8 audit tables accumulate genuine access telemetry in exactly the schema of the training features, enabling recalibration and retraining on production data as recommended in Section 7.4. This limitation is inherent to the absence of any public benchmark of application-layer decryption decisions (Section 2.3) and is revisited among the limitations in Section 7.3.

4.10 Chapter Summary

On a quarantined 1,500-sample test set the framework’s classifier achieved 0.9780 accuracy and 0.9777 weighted F1, passing 99.14% of legitimate requests, intercepting 92.8% of attacks outright, and never hard-blocking borderline users. The transpiled model reproduced the trained behavior faithfully at ≈ 50 microseconds per in-process inference, and the integrated system returned full plaintext, a masked view, or a zero-decryption denial for the same endpoint purely as a function of runtime behavior. Chapter V examines the engineering, ethical, and professional dimensions of deploying such a system.

CHAPTER V

Engineering, Ethical, and Professional Considerations

A framework that gates access to encrypted personal and financial data cannot be judged on algorithmic performance alone; it must be examined for engineering soundness, intellectual property, ethics, privacy and security, safety, legality, and societal and environmental consequences. This chapter considers each for the proposed adaptive decryption control framework specifically.

5.1 Introduction

These considerations matter because the framework operates directly on sensitive data (bank account and identity fields) and makes automated decisions that can withhold or expose that data. Engineering-wise, the system is modular: the encryption utilities, late-binding transformer, context builder, risk gatekeeper, and policy engine communicate through well-defined interfaces, so any component can be replaced independently. Reliability is supported by fail-safe defaults (the denial path exposes nothing) and by deterministic, reproducible behavior under a fixed seed. These design choices carry consequences beyond benchmark accuracy, examined below.

5.2 Intellectual Property Considerations

The synthetic dataset, the labeling-rule system, the late-binding transformer, the feature-mapping scheme, and the graduated policy engine are original work developed under supervision. Ideas drawn from prior work — the general Random Forest formulation, the AES-256-GCM construction, and the model-transpilation approach — are cited in Chapters II and III and extended rather than reproduced. All third-party software (NestJS, TypeORM, scikit-learn, m2cgen) is used under its respective open-source license; no proprietary code or data was incorporated, and any future commercialization would verify each dependency’s license terms.

5.3 Ethical Considerations

An automated system that can deny data access raises fairness and accountability questions. Three design properties address them. First, explainability: because the model is a Random Forest with published feature importances and every decision is logged with its

exact feature vector and class probabilities, any specific verdict can, in principle, be explained to an affected user or auditor — unlike an opaque black-box control. Second, fairness by construction: the features are behavioral and role-based, not demographic; the model never observes protected personal attributes. Third, honest reporting: this thesis reports both favorable results and limitations (the 9 undetected low-and-slow attacks, the reliance on synthetic data), with all statistics taken from seeded, reproducible runs. A production deployment should nonetheless monitor false-denial rates across user segments and provide a human appeal path for contested denials.

5.4 Data Privacy and Security Considerations

Privacy is the framework’s central purpose, and its design embodies data-minimization and defense-in-depth: sensitive fields are AES-256-GCM-encrypted before storage; ciphertext survives retrieval under late binding; decryption is gated and, on suspicion or attack, partial or absent; and the highest-value identifiers are masked without decryption whenever risk is elevated. The behavioral features are computed from server-side evidence only, so the scoring process itself introduces no new client-trusted inputs. Residual considerations for production include: strict custody and rotation of the encryption key (which must never reside in the database or source control); protection of the audit tables, which themselves record access patterns and must be access-controlled and retention-limited; and authenticated, plausibility-checked feature sources, since a system that trusts forged behavioral signals could be manipulated into over-exposing data.

5.5 Safety Considerations

The primary safety property is fail-safe exposure: the denial path performs zero decryption, so a classification error toward HIGH withholds data (a recoverable inconvenience) rather than leaking it (an unrecoverable breach). The residual safety risk is the opposite error — a true attack classified as LOW (9 of 139 in testing) — which the layered architecture mitigates because static RBAC, encryption at rest, and audit logging remain in force beneath the adaptive layer. As an offline evaluation, no decision here has governed real customer data; a production rollout should be staged (shadow-mode scoring, then masking, then denial) with monitoring at each stage.

5.6 Legal Considerations

Systems processing financial and identity data are subject to data-protection and sector-specific regulations. The framework supports compliance through encryption of personal data, minimization of plaintext exposure, and comprehensive audit logging suitable for accountability requirements; however, this thesis does not claim certification against any specific legal regime, and a deployment would require validation against applicable data-protection law and financial-sector regulation, plus institutional review for any trial involving real personal data.

5.7 Societal, Health, and Cultural Impact

Data breaches of financial and identity records impose direct social harm — fraud, identity theft, and erosion of trust in digital services. A framework that limits the blast radius of a compromised account or a malicious insider, by exposing only what runtime behavior justifies, contributes to safer digital-financial participation. Because the control is graduated rather than binary, it also avoids the societal cost of over-blocking legitimate users, preserving access to essential financial services for the 99%+ of benign requests. The approach is domain-general and applies equally to healthcare records and government data, where the confidentiality of personal information carries comparable social weight.

5.8 Environmental and Sustainability Considerations

The framework’s computational footprint is deliberately small: transpiled in-process inference (≈ 50 microseconds, no separate service) avoids the continuous energy cost of a dedicated inference server or the per-request overhead of a network round trip, and the training pipeline is a one-time cost over a 10,000-row dataset. By adding negligible per-request computation rather than provisioning always-on inference infrastructure, the design aligns with sustainable-computing goals; the dominant environmental consideration for a deployment would be the underlying database and application hosting, which the framework does not materially change.

5.9 Summary

The framework is modular and fail-safe; built on original data and algorithms with properly licensed dependencies; explainable and behaviorally (not demographically) fair; privacy-enhancing by purpose and design, with clear production hardening requirements

around key custody and audit-table protection; fail-safe toward withholding rather than leaking; consistent with but not certified against data-protection law; socially beneficial in limiting breach impact without over-blocking; and computationally lightweight. These considerations inform the limitations and future work of Chapter VII.

CHAPTER VI

Complex Engineering Problems and Activities

This chapter maps the thesis to the Complex Engineering Problem (CEP) and Complex Engineering Activity (CEA) attributes used to assess undergraduate engineering theses under the Washington-Accord-aligned framework.

6.1 Introduction

Building an adaptive decryption control framework required integrating cryptography, machine learning, backend software engineering, and security-domain modeling into one coherent system while managing genuine trade-offs — attack interception versus operational friction, model expressiveness versus transpilation size, and security strength versus per-request latency. These integrative and conflicting demands place the work firmly in the complex-engineering category rather than a routine application of a single technique.

6.2 Complex Engineering Problems Associated with the Project

Table 6.1: Mapping of the thesis to Complex Engineering Problem (CEP) attributes.

Attribute	Description	Relevance to this thesis
P1	Depth of knowledge required	Requires authenticated cryptography (AES-256-GCM, IV/tag discipline), supervised machine learning (Random Forest, evaluation methodology), stochastic data modeling (Poisson/exponential distributions), and backend/ORM software engineering.
P2	Conflicting requirements	Maximizing attack interception conflicts with minimizing false denials of legitimate users; model expressiveness conflicts with transpiled-artifact size; security checks conflict with latency. The graduated policy and the 75-tree/depth-12 choice explicitly manage these trade-offs.
P3	Depth of analysis required	Requires deriving a feature

		scheme, a labeling-rule system, and a risk-to-policy mapping, then interpreting per-class precision/recall, a confusion matrix, and feature importances against a threat model.
P4	Familiarity of issues	Combines a familiar algorithmic core (Random Forest, AES) with the non-standard idea of gating decryption itself on a learned per-request score via late binding — not a textbook pattern.
P5	Extent of applicable codes	Engages cryptographic best practice (authenticated encryption, IV uniqueness, key custody) and data-protection principles, though formal regulatory certification was out of scope (Section 5.6).
P6	Extent of stakeholder involvement	Stakeholders include end users (whose data confidentiality and access continuity are at stake), security/operations teams (who consume the audit trail), and the academic supervisor who shaped the problem formulation.
P7	Interdependence	The audit tables feed the features, the features feed the model, the model feeds the policy, the policy gates decryption, and each decision is logged back to the audit tables — a closed loop whose behavior cannot be understood from any single module in isolation.

6.3 Complex Engineering Activities Associated with the Project

Table 6.2: Mapping of the thesis to Complex Engineering Activity (CEA) attributes.

Attribute	Description	Relevance to this thesis
A1	Range of resources	Integrates a Python ML pipeline, a TypeScript/NestJS backend, a PostgreSQL database, a transpilation toolchain, and standard development tools into one reproducible system.
A2	Level of interaction	Requires coordinating multiple

		modules (dataset generator, trainer, transpiler, transformer, context builder, gatekeeper, policy engine) across a Python–JavaScript boundary held consistent by the shared static feature mapping.
A3	Innovation	Introduces a novel combination of late-binding cryptography, learned per-request risk scoring, graduated masking, and in-process transpiled inference (Section 3.7).
A4	Consequences to society and the environment	A deployed version could reduce financial-data breach impact (Section 5.7) at negligible computational cost (Section 5.8), while a miscalibrated policy could withhold data from legitimate users — hence the explicit fairness and monitoring discussion.
A5	Familiarity	Combines familiar undergraduate techniques (classification, symmetric encryption) with thesis-specific mechanisms (late binding, zero-decryption masking) developed beyond standard coursework.
A6	Leadership and management	As a single-author thesis, the author planned and executed the full pipeline across two terms under periodic supervisory review (Figure 1.3).
A7	Consequences of failure	A defect in the transformer, feature mapping, or policy could leak plaintext or wrongly deny access; this is mitigated by fail-safe defaults, feature-parity and end-to-end verification, and the zero-decryption guarantee on the denial path.

6.4 Mapping of the Proposed Work with Complex Engineering Attributes

Table 6.3: Consolidated mapping of major system components to CEP/CEA attributes.

System component	Primary CEP/CEA attributes	Justification
Synthetic dataset & labeling rules (Sec. 3.2)	P1, P3, A3	Encodes domain-expert security logic as a reproducible, noise-controlled generative model.
Feature scheme & static mapping (Sec. 3.3)	P3, P7, A2	Bridges Python training and JavaScript runtime through an invariant, interdependent encoding.
Random Forest & transpilation (Sec. 3.4)	P1, P2, A3	Balances expressiveness against artifact size; innovates by compiling the model into the request path.
Late-binding ALE (Sec. 3.4.4)	P2, P4, A3, A7	Inverts the decrypt/authorize ordering; correctness is safety-critical and independently verified.
Policy engine & masking (Sec. 3.4.5)	P2, P6, A4	Turns a risk tier into a graduated, stakeholder-visible data-exposure decision.
End-to-end verification (Sec. 4.5.4)	A7	Mitigates failure consequences by confirming the zero-decryption denial property on real ciphertext.

6.5 Summary

The thesis engages substantively with all seven CEP attributes — most centrally P2 (the interception-versus-friction and expressiveness-versus-size trade-offs) and P7 (the closed-loop interdependence of audit, features, model, and policy) — and all seven CEA attributes, most centrally A3 (the novel combination of late binding, learned scoring, masking, and transpiled inference) and A7 (managing the consequences of failure through fail-safe defaults and verification). It therefore exercises the depth, analysis, and judgment characteristic of a complex engineering problem rather than a routine application of an established algorithm.

CHAPTER VII

Conclusion and Future Work

This chapter restates the study, consolidates its major findings, acknowledges its limitations honestly, and recommends concrete directions for future work.

7.1 Summary of the Study

This thesis addressed the data-in-use vulnerability of modern application systems: the unconditional decryption of encrypted fields the moment an authenticated request arrives, before any behavioral assessment can occur. It proposed and built a machine learning-driven adaptive data access and decryption control framework that re-engineers the ORM retrieval path so ciphertext survives hydration (late-binding cryptography), extracts a behavioral feature vector from server-controlled evidence, scores each request with a transpiled 75-tree Random Forest executing in-process, and enforces one of three graduated cryptographic policies — full decryption, partial masking, or zero-decryption denial. The framework was implemented in a NestJS/PostgreSQL backend with a reproducible Python training pipeline and evaluated both offline and end-to-end.

7.2 Major Findings

- The risk classifier achieved 0.9780 accuracy and 0.9777 weighted F1 on a quarantined 1,500-sample test set, versus ≈ 0.75 –0.81 for a linear baseline — a ≈ 17 -point gain confirming that the security logic is conditional and non-linear.
- Operationally, the system passes 99.14% of legitimate requests with full decryption, intercepts 92.8% of attack vectors outright, and never hard-blocks a borderline (MEDIUM-tier) user.
- The learned feature-importance ranking (velocity, sensitivity, authentication friction, ownership dominating) independently reproduces the threat model, evidencing genuine domain logic rather than dataset artifacts.
- The transpiled model reproduces the trained decisions faithfully and executes in ≈ 50 microseconds in-process; total per-request security overhead is ≈ 2 ms, dominated by the audit insert and feature queries, not by inference.
- End-to-end, the same endpoint returns full plaintext, a masked view (with the primary identifier masked using zero decryption), or an HTTP 403 purely as a function of

runtime behavior; the denial path performs zero application-level decryption calls (verified end-to-end), so a correct HIGH classification prevents exposure rather than detecting it after the fact.

7.3 Limitations of the Study

- Synthetic training data. The corpus is generated under expert rules with controlled noise rather than harvested from production traffic; absolute behavior against real-world access patterns is not independently validated. Because the labels themselves are rule-generated, the classifier may partly be re-learning the labeling policy rather than generalizable malicious behavior (Section 4.9), so the reported metrics constitute proof-of-concept validation rather than evidence of real-world detection performance.
- Low-and-slow evasion. 9 of 139 test attacks — scrapers pacing below the velocity thresholds — evade detection, an inherent blind spot of per-request velocity features.
- Single-node prototype. Behavioral counts are stored in PostgreSQL and are correct across application replicas, but very high-traffic multi-node deployment (e.g., Redis-backed counters) was not evaluated.
- Fixed windows and thresholds. The 60-second velocity window, 24-hour failure window, and the risk-tier boundaries baked into the trained forest were not swept or auto-tuned.
- Single demonstrated domain. One high-sensitivity aggregate (bank details) demonstrates the pattern; the reusable module was not exercised across multiple domains or list/batch endpoints.
- Offline evaluation only. The framework was not integrated into a live system or trialed with real users.

7.4 Future Recommendation

1. Replace or calibrate the synthetic corpus with real, privacy-preserving access telemetry accumulated by the audit layer, closing the loop from synthetic bootstrap to production-data retraining.
2. Add longer-horizon aggregate features (records touched per hour or day, sequence models over access history) to catch low-and-slow scraping that per-request velocity misses.

3. Introduce calibrated-probability thresholds and a configurable policy engine so risk-tier boundaries can be changed without retraining.
4. Move behavioral counters to a distributed store (e.g., Redis sliding windows) and evaluate multi-node, high-throughput deployment.
5. Generalize the adaptive module across additional domains (healthcare, e-commerce) and to list/batch endpoints with per-record or min-aggregated scoring.
6. Enrich the feature set with device fingerprint, geolocation, and session age, and study fairness of denial rates across user segments.
7. Conduct a staged field trial (shadow scoring, then masking, then denial) with monitoring and a human appeal path for contested denials.

7.5 Closing Statement

This thesis demonstrates that the decryption boundary itself can be made adaptive: that an application can decide, per request and from unforgeable runtime evidence, how much encrypted data to reveal — and that it can withhold data by never invoking application-level decryption at all, rather than detecting a breach after it. By combining late-binding cryptography, a learned and auditable behavioral risk model, a graduated masking policy, and dependency-free in-process inference, the framework addresses a gap left open by static encryption and static access control alike, achieving high attack interception and near-zero legitimate friction at negligible latency. While synthetic training data and the absence of field validation leave clear work ahead, the consistent, verified results provide a solid foundation for adaptive, data-in-use protection in secure application systems built on encrypted databases.

References

- [1] V. Shaik and K. Natarajan, “Flexible and cost-effective cryptographic encryption algorithm for securing unencrypted database files at rest and in transit,” *MethodsX*, vol. 9, art. no. 101924, 2022, doi: 10.1016/j.mex.2022.101924.
- [2] V. Sidorov and W. K. Ng, “Transparent Data Encryption for Data-in-Use and Data-at-Rest in a Cloud-Based Database-as-a-Service Solution,” in *Proc. 2015 IEEE World Congress on Services (SERVICES)*, 2015, doi: 10.1109/SERVICES.2015.40.
- [3] E. F. Khalaf and M. M. Kadi, “A Survey of Access Control and Data Encryption for Database Security,” *JKAU: Engineering Sciences*, vol. 28, no. 1, pp. 19–30, 2017, doi: 10.4197/Eng.28-1.2.
- [4] N. Sharma and P. Yadav, “Efficient Implementation of AES-256 for Secure Machine Learning Datasets: A Performance and Compatibility Study,” *International Journal of Engineering Trends and Technology*, vol. 73, no. 9, pp. 91–99, 2025, doi: 10.14445/22315381/IJETT-V73I9P109.
- [5] R. Prajapati, S. S. Panchal, N. Soni, and S. R. Panchal, “Machine Learning-Based Risk-Adaptive Attribute-Based Access Control Framework for Secure IoT Networks,” *International Research Journal of Multidisciplinary Technovation*, vol. 8, no. 3, pp. 495–519, 2026, doi: 10.54392/irjmt26329.
- [6] A. Liu, X. Du, and N. Wang, “Efficient access control permission decision engine based on machine learning,” *Security and Communication Networks*, vol. 2021, art. no. 3970485, 2021, doi: 10.1155/2021/3970485.
- [7] V. K. Matter, M. G. Martins, and J. L. V. Barbosa, “Context-aware security and machine learning for access control: A systematic mapping and taxonomies,” *Computer Science Review*, vol. 60, art. no. 100880, 2025, doi: 10.1016/j.cosrev.2025.100880.
- [8] M. Usman, M. S. Sarfraz, U. Habib, M. U. Aftab, and S. Javed, “Automatic hybrid access control in SCADA-enabled IIoT networks using machine learning,” *Sensors*, vol. 23, no. 8, art. no. 3931, 2023, doi: 10.3390/s23083931.

- [9] K. A. Abuhasel, "A Zero-Trust Network-Based Access Control Scheme for Sustainable and Resilient Industry 5.0," *IEEE Access*, vol. 11, pp. 116398–116409, 2023, doi: 10.1109/ACCESS.2023.3325879.
- [10] B. Thuraisingham, "Database Security: Past, Present, and Future," in *Proc. IEEE International Congress on Big Data*, 2015, pp. 772–774.
- [11] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 6th ed. USA: Pearson Education, 2011.
- [12] H. Pourzargham, "Importance of Security in Database," *International Journal of Computer Science and Network Security*, vol. 15, no. 5, pp. 29–32, 2015.
- [13] D. Servos and S. L. Osborn, "Current Research and Open Problems in Attribute-Based Access Control," *ACM Computing Surveys*, vol. 49, no. 4, pp. 1–45, 2017.
- [14] P. K. Arya, M. Aswal, and V. Kumar, "Comparative Study of Asymmetric Key Cryptographic Algorithms," *International Journal of Computer Science and Communication Networks*, vol. 5, no. 1, pp. 17–21, 2015.
- [15] G. Singh and Supriya, "A Study of Encryption Algorithms (RSA, DES, 3DES and AES) for Information Security," *International Journal of Computer Applications*, vol. 67, no. 19, pp. 33–38, 2013.
- [16] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Transactions on Information and System Security*, vol. 4, no. 3, pp. 224–274, Aug. 2001. doi: 10.1145/501978.501980.
- [17] OASIS, "WS-Security Policy 1.2," OASIS Standard, Nov. 2007. [Online]. Available: <https://docs.oasis-open.org/ws-sx/ws-securitypolicy/v1.2/>
- [18] B. Schneier, *Applied Cryptography: Protocols, Algorithms, and Source Code in C*, 2nd ed. New York: Wiley, 1996.
- [19] IEEE Standards Association, "IEEE Standard for Information Technology—POSIX Base Specifications," IEEE Std 1003.1-2017, 2018. doi: 10.1109/IEEESTD.2018.8277153.
- [20] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001. doi: 10.1023/A:1010933404324.